

PHASE 1 COMPLETE REVIEW

Phase 1 리뷰

Webhook 202 패턴 · Redis Stream · TxStateMachine · LedgerService
ChainEventListener · 통합 테스트 10개 시나리오

issuer-service

vasp

event-engine

core-banking

chain-adapters

COINCRAFT
ACADEMY

전체 아키텍처 맵

ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · M3는 레이어 3 비즈니스 로직

내부망

레이어 1 — 사용자

Kyobo App

↓ 사용자 이벤트 / API 응답

레이어 2 — 게이트웨이

Webhook
Receiver

Redis Event
Pipeline

IVASP
Adapter

IBlockchain
Adapter

↓ 발행 요청 / 원장 업데이트

레이어 3 — 코어 서비스

비즈니스 로직

데이터 관리

거버넌스

인터넷

레이어 5 — 퍼블릭 블록 체인

이더리움 EVM

↓ Tx Broadcast / 이벤트 감지

레이어 4 — 인가 VASP

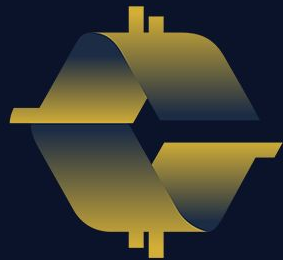
인가 VASP

↑ 발행 요청 / Webhook 이벤트 ↓

M3 핵심 구간: 레이어 2 TX 상태머신 — VASP에 TX를 보낸 후 블록체인이 어떤 경로로 실패하는가, 각 상태(REQUESTED→CONFIRMED→FINALIZED)를 어떻게 추적·복구하는가

event-engine

WebhookServer · RedisStreamPublisher · ConsumerGroupWorker ·
NFTIssuedProcessor · DLQHandler



COINCRAFT
ACADEMY

Phase 1 — Webhook 발신자 구분

WHO SENDS WEBHOOKS TO WebhookReceiver IN PHASE 1

[Phase 1 — 현재 구현] 이 모듈은 VASP(월렛원) 위탁 아키텍처를 기반으로 합니다.

ACTIVITY_ACHIEVED

- 발신자: 내부망 (교보 앱 서버)
- 사용자 활동 달성 알림
- WebhookReceiver가 수신
- → Queue 적재 → NFT 발행 워크플로우

NFT_ISSUED

- 발신자: 월렛원 (외부 VASP)
- 온체인 TX 확정 후 발행 완료 콜백
- WebhookReceiver가 수신
- → 원장 업데이트 트리거

Phase 3 예고: 직접 Custody 운영 시 NFT_ISSUED는 VASP Webhook이 아니라 ChainEventListener가 블록체인 이벤트를 직접 구독해 처리한다.

이 세션의 구간

SESSION SCOPE — FROM APP SERVER TO QUEUE ENTRY

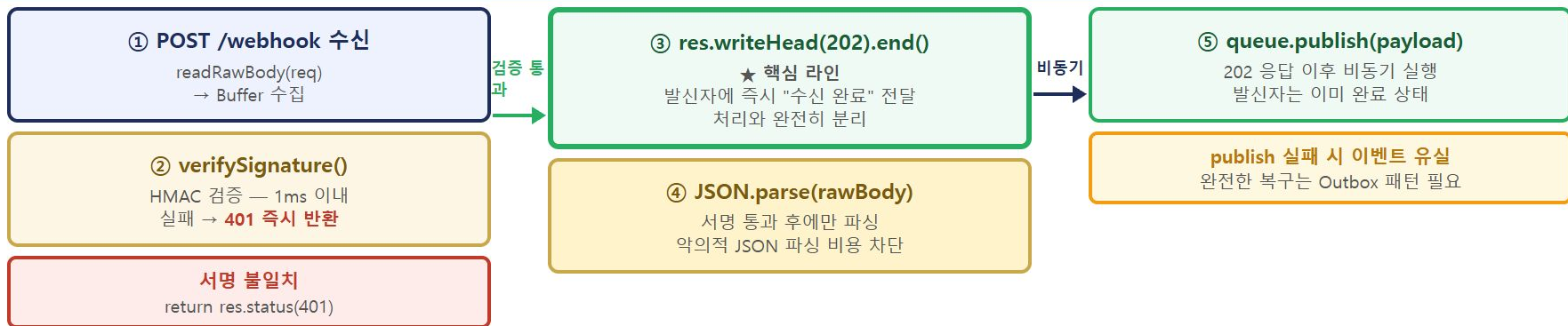


중요: 블록체인 이벤트 (NFT 발행 확인)는 WebhookServer가 아닌 ChainEventListener가 수신한다.

Core Banking은 내부망이 아웃바운드로 알림을 보내는 대상 — WebhookServer를 호출하지 않는다.

1-2. 올바른 방식 — 202 + Queue 패턴 — 실행 흐름

RECEIVE → VERIFY → 202 IMMEDIATELY → QUEUE ASYNC: DECOUPLED RESPONSIBILITY



핵심: 202 응답과 Queue 적재는 시간 순서가 분리된다 — 수신 책임과 처리 책임이 독립적으로 각자 실패·복구 가능하다.

2부 — WebhookServer 클래스 구조 전체 읽기

2-1. CLASS STRUCTURE — TYPES, CONSTRUCTOR, PUBLIC API

```
export type WebhookPayload = {
  eventType: string;
  data:      Record<string, unknown>;
  timestamp: number;
  requestId: string;    // 멍등성 키 - 중복 처리 방지에 사용
};

export type WebhookHandler = (payload: WebhookPayload) => Promise<void>;

export class WebhookServer {
  private server: http.Server;
  private handlers: Map<string, WebhookHandler[]> = new Map();

  constructor(private readonly config: {
    port:      number;
    secret:    string;    // HMAC 시크릿 - 환경 변수로 주입
    maxBodyKb: number;    // payload 크기 제한
  }) {
    this.server = http.createServer(this._handleRequest.bind(this));
  }

  // 핸들러 등록 - 체이닝 가능
  on(eventType: string, handler: WebhookHandler): this { ... }
  listen(): Promise<void> { ... }
  close(): Promise<void> { ... }

  private async _handleRequest(req, res): Promise<void> { ... } // ← 이 세션
  private      _readBody(req):      Promise<Buffer> { ... } // ← 이 세션
  private      _verifySignature(rawBody, sig): boolean { ... } // ← S6 상세
}
```

2부 — WebhookServer 클래스 구조 전체 읽기 — 실행 흐름

CLASS ANATOMY: TYPES → CONSTRUCTOR → PUBLIC API → PRIVATE METHODS

타입 계층

① WebhookPayload

eventType · data · timestamp

requestId = 멍등성 키

중복 처리 방지에 사용

② WebhookHandler

(payload) → Promise<void>

비동기 함수 시그니처

핸들러 타입 계약

클래스 상태

③ private server

http.Server 인스턴스

constructor에서 생성

_handleRequest 바인딩

④ private handlers

Map<eventType, Handler[]>

같은 이벤트에 N개 핸들러

체이닝으로 등록

메서드 계층

⑤ Public: on / listen / close

on() → this 반환 (체이닝)

listen() → 포트 바인딩

⑥ Private: _handleRequest

★ 이 세션 핵심

요청 전체 처리 파이프라인

⑦ Private: _readBody / _verifySignature

Buffer 수집 · HMAC 검증

핵심: 타입 → 상태 → Public API → Private 구현 순으로 읽으면 클래스 전체 책임이 한눈에 보인다. private 메서드 3개가 이 세션의 전부다.

핸들러 등록 — 체이닝 패턴 — 실행 흐름

HANDLER REGISTRATION → MAP STORAGE → EVENT DISPATCH → ALLSETTLED EXECUTION

① server.on('NFT_ISSUED', h1)

handlers.get('NFT_ISSUED') → undefined
→ new Map entry: ['NFT_ISSUED', [h1]]
return this → 체이닝 가능

② .on('NFT_ISSUED', h2)

handlers.get('NFT_ISSUED') → [h1]
→ push(h2): ['NFT_ISSUED', [h1, h2]]
같은 이벤트에 누적

③ handlers Map 내부 상태

'NFT_ISSUED' → [h1(Queue적재), h2(AuditLog)]

'NFT_BURNED' → [h3(Queue적재)]

④ eventType 수신

payload.eventType으로 Map 조회
없으면 [] → 아무것도 안 함

⑤ handlers.map(h→h(payload))

Promise 배열 생성
h1, h2 동시 시작

⑥ Promise.allSettled()

h2 실패해도 h1 결과 유지
각 독립 결과 반환

핵심: on()은 Map에 핸들러를 누적 등록하고 this를 반환해 체이닝을 가능하게 한다. 이벤트 수신 시 해당 eventType의 모든 핸들러가 독립 실행된다.

RedisStreamPublisher.publish() — 실행 흐름

INPUT → TYPE COERCION → XADD → MESSAGE ID RETURN

① publish(event: StreamEvent) 호출

streamKey / eventType / payload(object) / txHash / blockNumber(number) / requestId



② 타입 변환 (string-string map 제약)

payload: object (저장 불가)

JSON.stringify() → string ✓

blockNumber: number (저장 불가)

String(blockNumber) ✓



③ redis.xadd(streamKey ?? defaultStream, fields)

id='*' 자동생성 · Redis append-only log에 적재



④ return messageid ("1714000000000-0")

호출자가 DB에 저장 → At-least-once 추적용 키

StreamEvent 인터페이스

streamKey: string (기본값: defaultStream)

eventType: string

payload: Record<string,unknown> → JSON 직렬화

txHash: string

blockNumber: number → String() 변환

requestId: string (역등성 키)

?? 연산자: streamKey가 null/undefined이면 this.defaultStream 사용

publishedAt: String(Date.now()) — 발행 시간 ms

핵심: Redis의 string-string 제약 때문에 object→JSON, number→String 변환이 필수다. 이를 빠뜨리면 런타임 에러가 발생한다.

정상 처리 흐름 — 실행 흐름

XREADGROUP DELIVERS → PEL RECORDS OWNERSHIP → XACK CLEARS

① XREADGROUP GROUP ... >

미처리 새 메시지 요청 · BLOCK 5000ms 대기



② PEL 등록

{ msg-001: consumer-1, idleTime:0, deliveryCount:1 }



③ processor.process(message)

LedgerService.update() 등 비즈니스 로직 실행



④ XACK → PEL 제거

PEL: {} — 처리 완료 확정 ✓

PEL 상태 변화

Step 1 후: **PENDING** (deliveryCount=1)

Step 3 중: 계속 PENDING (idleTime 증가)

Step 4 후: **제거** → 완료

Step 3에서 crash 발생 시

XACK 없음 → PEL에 잔류

idle 초과 → XAUTOCLAIM → 재배달

deliveryCount: 1 → 2

핵심: XREADGROUP이 PEL에 소유권을 등록하고, XACK만이 그것을 제거한다. 이 두 명령이 At-least-once의 전부다.

장애 복구 흐름 — XAUTOCLAIM

CONSUMER CRASH RECOVERY: PEL IDLE TIMEOUT → OWNERSHIP TRANSFER

1 XREADGROUP ... >
msg-002 반환 · PEL: consumer-1 | idleTime: 0

2 consumer-1 [crash!]
XACK 없음 → PEL에 msg-002 그대로 남음

3 ⌚ 30,000ms 경과 — minIdleMs 초과

4 consumer-2 XAUTOCLAIM 호출
msg-002 발견 → 소유권 이전: consumer-2 | deliveryCount: 2

5 consumer-2 처리 → XACK ✓
PEL 비어있음 → at-least-once 완료

XAUTOCLAIM 명령어

```
XAUTOCLAIM kyobo:events issuer-consumers  
consumer-2 30000 0-0 COUNT 10  
# 30초(30000ms) 이상 idle인 PEL 메시지를  
# consumer-2에게 자동 이전 (최대 10개)
```

minIdleMs 설정 기준

- 처리 시간의 **3~5배**를 minIdleMs로 설정
- 너무 짧게 설정 → 처리 중인 메시지가 다른 Consumer에 재배포
- 멍등성이 있으면 안전하지만 불필요한 중복 처리 증가

Redis 7.0+: XAUTOCLAIM 권장
구버전: XPENDING → XCLAIM 조합

HMAC 원리 — 송수신 대칭 구조

HASH-BASED MESSAGE AUTHENTICATION CODE

발신자 (교보 앱 서버)

```
secret = "kyobo-webhook-secret-2024"  
payload = '{eventType:"ACTIVITY_ACHIEVED",...}'  
  
sig = HMAC-SHA256(secret, payload)  
→ "a3f4b2c1d8e9f0..." (64자 hex)
```

HTTP 요청:

X-Kyobo-Signature: a3f4b2c1d8e9f0...

Body: {eventType:"ACTIVITY_ACHIEVED",...}



수신자 (WebhookServer)

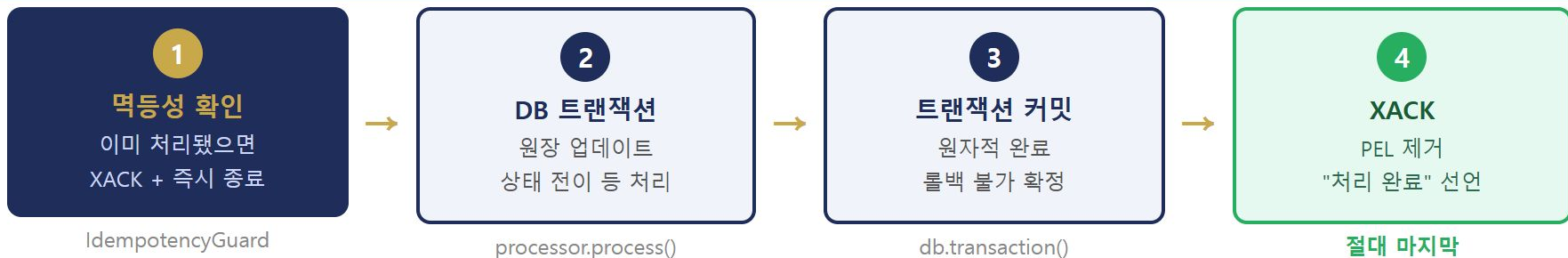
```
rawBody = 수신한 Body bytes (Buffer 그대로)  
expected = HMAC-SHA256(secret, rawBody)  
← 발신자와 동일한 계산  
  
timingSafeEqual(received, expected)
```

일치 → 발신자 인증 + 무결성

불일치 → 위조된 요청

올바른 처리 순서 — 4단계 불변 규칙

THE FOUR-STEP INVARIANT — NEVER CHANGE THIS ORDER



불변 규칙 — XACK는 반드시 Step 4. DB 커밋 성공을 확인한 후에만 실행한다. 이 순서는 절대 바꾸지 않는다.

코드로 보는 4단계 — `_handleWithRetry()`

ConsumerGroupWorker.ts — THE FOUR-STEP INVARIANT IN CODE

```
private async _handleWithRetry(msg: StreamMessage): Promise<void> {
  const retryCount = parseInt(msg.fields['_retryCount'] ?? '0', 10);

  // — ① 멍등성 확인 + ② DB 트랜잭션 + ③ 커밋 —————
  // processor.process() 내부에서 IdempotencyGuard.run() + db.transaction() 처리
  try {
    await Promise.all(
      this.processors
        .filter(p => p.eventTypes.includes(msg.fields['eventType'] ?? ''))
        .map(p => p.process(msg))
    );

    // — ④ XACK - 절대 마지막 —————
    await this.redis.xack(this.config.streamKey, this.config.groupName, msg.id);

  } catch (err) {
    // DB 실패 → XACK 실행하지 않음 → PEL 잔류 → 재시도
    msg.fields['_retryCount'] = String(retryCount + 1);
  }
}
```

핵심 — try 블록 안에서 processor.process()가 ①②③을 담당, 성공 후에만 XACK(④)가 실행된다. catch에서 XACK가 없으므로 실패 시 5 / 108

코드로 보는 4단계 — 단계별 해설

ConsumerGroupWorker.ts — WHAT EACH STEP DOES INTERNALLY

① ② ③ — processor.process() 내부

- ① **IdempotencyGuard.run()** 호출
→ 이미 처리된 requestId면 callback 건너뛴
- ② **db.transaction(trx => { ... })**
→ user_nft_holdings INSERT
→ ON CONFLICT DO NOTHING (멥등성)
- ③ **트랜잭션 커밋 (자동)**
→ 원자적 완료 확정

④ XACK — 절대 마지막

- try 성공 후에만 실행
- PEL에서 메시지 제거
- "처리 완료" 공식 선언

catch — XACK 없음

- PEL 잔류 → minIdleMs 후 재수신
- _retryCount 누적
- maxRetries 초과 시 DLQ

Consumer 수평 확장과 멱등성

HORIZONTAL SCALING — IDEMPOTENCY MAKES IT SAFE

Consumer Group 메시지 분배 방식

Consumer-1

메시지 A, D, G

Consumer-2

메시지 B, E, H

Consumer-3

메시지 C, F, I

XREADGROUP: 먼저 호출한 Consumer가 메시지 소유

→ 같은 메시지를 두 Consumer가 동시에 처리 불가 (정상 상황)

XAUTOCLAIM 예외 상황

Consumer-1 장애 → 메시지 A PEL 잔류

→ Consumer-2가 XAUTOCLAIM으로 탈취

→ 멱등성 확인 → 이미 처리? 아니면 재처리

→ 어느 쪽이든 정합성 유지

멱등성 키

온체인: txHash + logIndex

Webhook: requestId

DB: processed_events UNIQUE

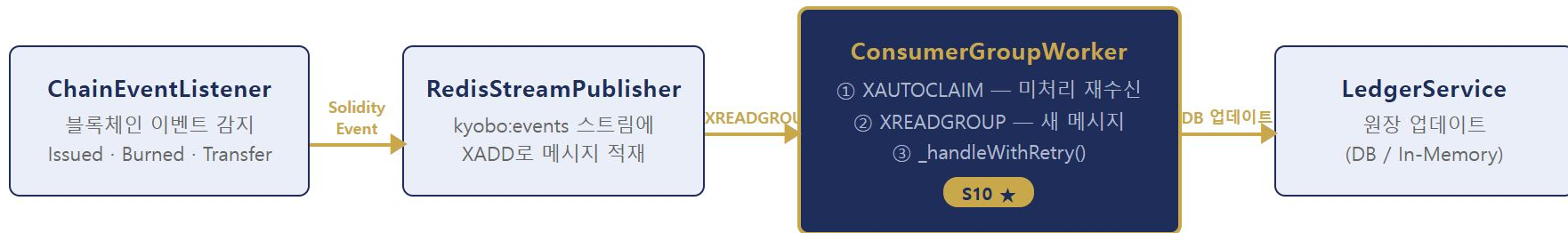
→ ON CONFLICT DO NOTHING

→ 중복 삽입 시 조용히 무시

결론 — Consumer를 N배 늘려도 멱등성이 중복 처리를 차단한다. 수평 확장과 정합성이 공존한다.

ConsumerGroupWorker — 파이프라인에서의 위치

POSITION IN THE EVENT PIPELINE



S10의 역할 — 블록체인 이벤트와 원장 DB 사이 신뢰성 보장 계층. 재시도·멥등성·DLQ가 모두 이 Worker 안에 구현된다.

EventProcessor 인터페이스 — `eventTypes: string[] + process(msg): Promise<void>` · `process()`는 반드시 멥등성 보장 필수

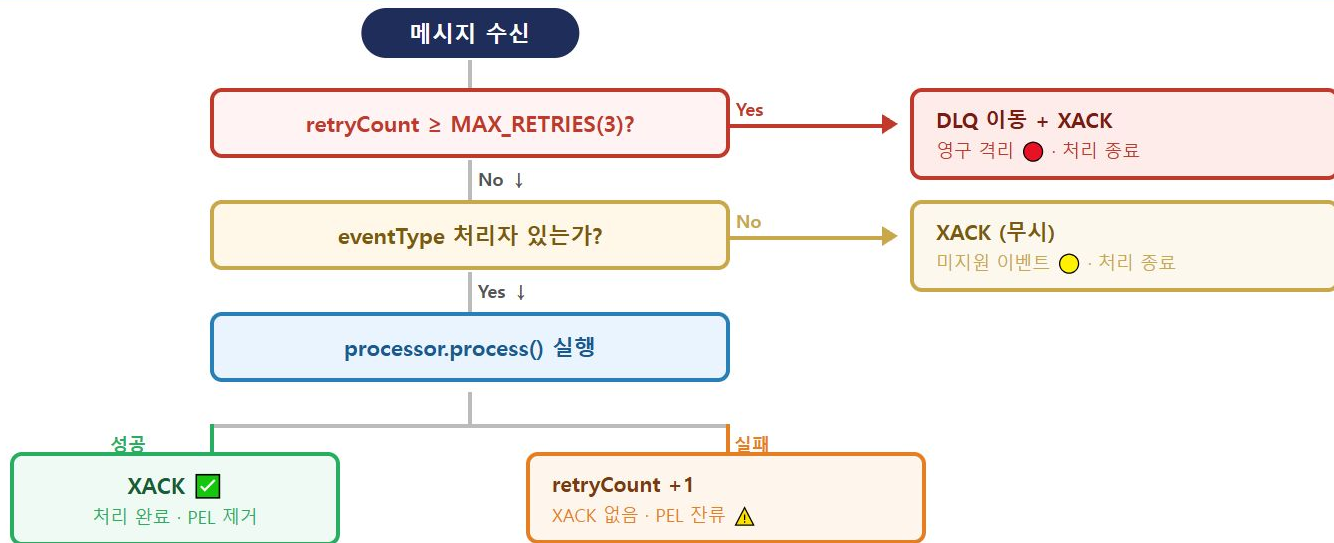
ConsumerGroupWorker — 의존 관계

CONSTRUCTOR DEPENDENCIES — THREE COLLABORATORS + CONFIG



메시지 1건의 운명 — _handleWithRetry 결정 트리

DECISION TREE — EVERY MESSAGE PASSES THROUGH THIS FUNCTION



핵심 — XACK 안 하는 유일한 경우 = 처리 실패. 이것이 At-least-once 보장의 본체.

ConsumerGroupPool — 도메인별 Consumer Group 분리

DOMAIN ISOLATION — INDEPENDENT PEL · RETRY · DLQ PER GROUP

단일 그룹의 문제

activity 이벤트가 수만 건 쌓이면 → nft 이벤트도 같은 PEL 대기열 → **전체 도메인 지연**

ConsumerGroupPool 구조

kyobo:events (단일 스트림)

nft-consumers

NFTIssuedProcessor
PEL · DLQ 독립

activity-consumers

ActivityProcessor
PEL · DLQ 독립

coupon-consumers

CouponProcessor
PEL · DLQ 독립

각 그룹은 같은 스트림을 독립적으로 구독 — 오프셋 분리
모르는 eventType → processors.length === 0 → 즉시 XACK (PEL 오염 없음)

사용법 (ConsumerGroupPool.ts)

```
// issuer-service/src/index.ts - 실제 wiring
const pool = new ConsumerGroupPool(redisAdapter, streamDlq,
  { streamKey: 'kyobo:events', batchSize: 10,
    blockMs: 5_000, minIdleMs: 30_000 },
  [
    { groupName: 'nft-consumers',
      consumerId: 'nft-1',
      processors: [nftIssuedProcessor] },
    // 추가 예시 (주석 해제만으로 활성화)
    // { groupName: 'activity-consumers', consumerId: 'activity-1',
    //   processors: [activityProcessor] },
    // { groupName: 'coupon-consumers', consumerId: 'coupon-1',
    //   processors: [couponProcessor] },
  ]
);
await pool.start();
```

격리 효과 — activity가 밀려도 nft-consumers PEL에 영향 없음. 장애-재처리 범위도 그룹 단위 분리.

발행 측 변경 없음 — 스트림은 하나. RedisStreamPublisher 수정 불필요.

NFTIssuedProcessor — EventProcessor 구현 패턴

IMPLEMENTING AN EVENT PROCESSOR WITH IDEMPOTENCY

구현체 패턴

```
import { EventType } from '../EventTypes';

class NFTIssuedProcessor implements EventProcessor {
  readonly eventTypes = [EventType.NFT_ISSUED];

  async process(msg: StreamMessage): Promise<void> {
    const event = JSON.parse(msg.fields['payload'] ?? '{}');
    const key = `NFTIssued:${event.txHash}:${event.logIndex}`;

    await this.idempotencyGuard.run(key, async () => {
      await this.ledgerService.db.transaction(async trx => {
        await trx('mint_requests').where({...})
          .update({ status: 'CONFIRMED' });
        await trx('user_nft_holdings')
          .insert({...}).onConflict().merge();
      });
    });
  }
}
```

완료 기준 (4가지 시나리오 검증)

- 시나리오 1 — retryCount ≥ MAX_RETRIES → DLQ 이동 ✅ + XACK ✅
- 시나리오 2 — processor 없음 → XACK ✅ (호출 없음)
- 시나리오 3 — 처리 성공 → XACK ✅
- 시나리오 4 — 처리 실패 → retryCount=2 ✅ + XACK 없음 ✅

역등성 키 — txHash + logIndex로 온체인 이벤트 고유 식별. 같은 이벤트 2번
와도 DB는 1번만 변경.

실행 — event-engine 폴더에서 `npx ts-node
src/exercises/S10_handle_with_retry.ts`

EventType 중앙화 + Processor 확장 구조

CENTRALIZED EVENT TYPES — COMPILE-TIME SAFETY + EXTENSIBILITY

EventTypes.ts — as const 패턴

```
export const EventType = {  
  // 온체인 이벤트  
  NFT_ISSUED: 'NFT_ISSUED',  
  NFT_BURNED: 'NFT_BURNED',  
  NFT_TRANSFERRED: 'NFT_TRANSFERRED',  
  // 활동 기반 이벤트  
  WALK_GOAL_MET: 'WALK_GOAL_MET',  
  HEALTH_CHECK_DONE: 'HEALTH_CHECK_DONE',  
  COUPON_CLAIM: 'COUPON_CLAIM',  
  CAMPAIGN_REWARD: 'CAMPAIGN_REWARD',  
} as const;  
export type EventType =  
  typeof EventType[keyof typeof EventType];
```

as const — 값이 string 리터럴 타입으로 좁혀짐. 오타 시 컴파일 에러로 즉시 탐지.

Processor 확장 구조

Processor	eventTypes	상태
NFTIssuedProcessor	NFT_ISSUED	완성
NFTBurnedProcessor	NFT_BURNED	stub
NFTTransferredProcessor	NFT_TRANSFERRED	stub
ActivityProcessor	WALK_GOAL_MET · HEALTH_CHECK_DONE · ACTIVITY_ACHIEVED	stub
CouponProcessor	COUPON_CLAIM · CAMPAIGN_REWARD	stub

새 이벤트 추가 3단계 ① EventTypes.ts 상수 추가 → ② Processor 구현 → ③ ConsumerGroupPool 그룹 등록

기존 코드 무변경 — 새 Processor 추가해도 ConsumerGroupWorker · RedisStreamPublisher 수정 불필요.

DLQHandler 구조 + DLQItem 인터페이스

DLQHANDLER.TS — STRUCTURE AND DATA CONTRACT

파일 위치 및 설정

📁 internal/packages/event-engine/src/stream/DLQHandler.ts

- DLQ 스트림 키: `kyobo:events:dlq`
- 3가지 메서드: `move()`, `listPending()`, `requeueMessage()`
- 알림 채널: `IAAlertNotifier` 인터페이스

```
// DLQHandler.ts:16
export interface DLQItem {
  messageId: string; // Redis messageId (예: "1714320000000-0")
  streamKey: string; // 원본 스트림 키
  groupName: string; // Consumer Group 이름
  event: Record<string, string>; // 원본 이벤트 필드 전체
  reason: string; // 실패 원인 문자열
  failedAt: Date;
}
```

3가지 메서드 역할

move(item)

DLQ 스트림에 XADD
운영팀 알림 발송
원 스트림 XACK

listPending(count)

DLQ XRANGE 조회
DLQItem[] 반환
운영자 모니터링용

requeueMessage(id)

DLQ 메시지 조회
원 스트림 재발행
DLQ에서 삭제

DLQ 운영 4단계 절차

OPERATOR RUNBOOK — FOUR STEPS FROM ALERT TO RESOLUTION



2회 DLQ = 비즈니스 로직 버그 의심 — 같은 메시지가 재처리 후 또 DLQ로 가면 `_requeuedFrom` 필드로 추적. 코드 수정 없이는 재투입 의미 없음.

FinalizedBlockProvider — VASP 연동에서 필요한가

FINALIZED BLOCK CHECK IN VASP-DELEGATED ARCHITECTURE

Phase별 역할 분담

Phase	TX 처리 주체	FinalizedBlockProvider
Phase 1	외부 VASP	미주입 — 스킵
Phase 2	외부 VASP + 직접 구독	선택적 활성화
Phase 3	Kyobo Custody (HSM/MPC)	필수 주입

Phase 1 트레이드오프 — VASP SLA가 finalization을 보장하면 이중 체크 불필요. 보장하지 않으면 Reorg 위험을 VASP에 위임.

optional 설계 이유 (?:)

```
// Phase 1 — FinalizedBlockProvider 미주입
// VASP 신뢰, finalization 체크 스킵
const processor = new NFTIssuedProcessor(
  idempotency, ledger
);

// Phase 3 — FinalizedBlockProvider 주입
// 직접 온체인 조회, 미확정 블록 → Deferred
const processor = new NFTIssuedProcessor(
  idempotency, ledger,
  evmAdapter // getFinalizedBlockNumber()
);
```

구현체 교체 비용 0 — 상위 레이어 수정 없이 생성자 인자 하나만 추가. Phase 1 → 3 전환이 이 줄 하나.

M2 실습 기본값 — FinalizedBlockProvider 미주입. Finalized 체크 없이 전체 파이프라인 E2E 동작 확인이 목표. Phase 3 전환 시 EVMAdapter 주입만으로 활성화.

NFTIssuedProcessor — 구조와 의존성

NFT_ISSUED EVENT PROCESSOR — THREE DEPENDENCIES

클래스 구조 (processors/NFTIssuedProcessor.ts)

```
export class NFTIssuedProcessor
  implements EventProcessor {

  readonly eventTypes = ['NFT_ISSUED'];

  constructor(
    private readonly idempotency:
      IdempotencyGuard,          // 필수
    private readonly ledger:
      LedgerService,             // 필수
    private readonly finalizedBlockProvider?:
      FinalizedBlockProvider,    // 선택
  ) {}
}
```

3개 의존성 상세

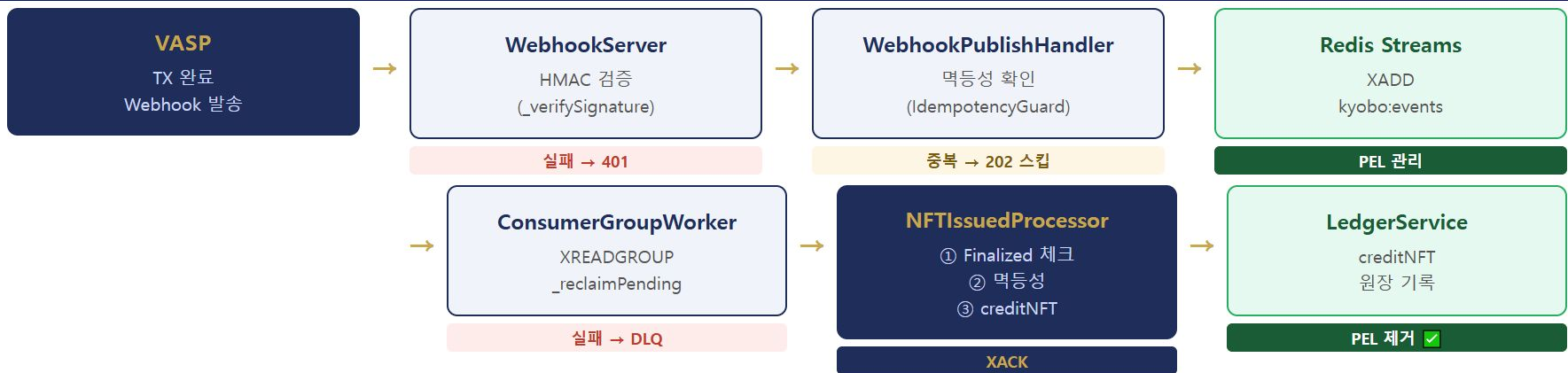
의존성	역할	필수?
IdempotencyGuard	중복 처리 차단 (run + key)	필수
LedgerService	원장 업데이트 (creditNFT)	필수
FinalizedBlockProvider	블록 확정 여부 조회	선택 (?:)

선택적 의존성 설계 이유 — Finalized 검증 없이도 동작 가능 (M2 실습 기본값).
점진적 기능 추가에 유리. 테스트에서 mock 교체 용이.

DIP 원칙 — 의존성이 모두 인터페이스 → M4에서 LedgerService를 PostgreSQL 구현체로 교체해도 이 클래스 코드 변경 0.

M2 파이프라인 전체 — 한눈에 보기

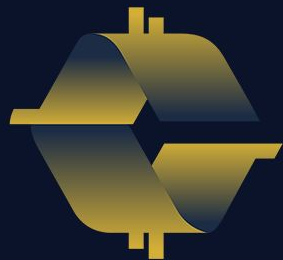
M2 FULL PIPELINE — END-TO-END REVIEW



M2 핵심 — 각 레이어는 인터페이스만 바라봄. WebhookServer는 Handler를 모르고, Worker는 Processor 내부를 모르고, Processor는 LedgerService 구현체를 모름.



TxStateMachineService · submitMintRequest · 재시도 · Circuit
Breaker · VaspTxClientAdapter



COINCRAFT
ACADEMY

TX 상태 8개 — 각 상태의 보장

TX STATUS DEFINITIONS · "이 상태에 있을 때 무엇이 참인가"

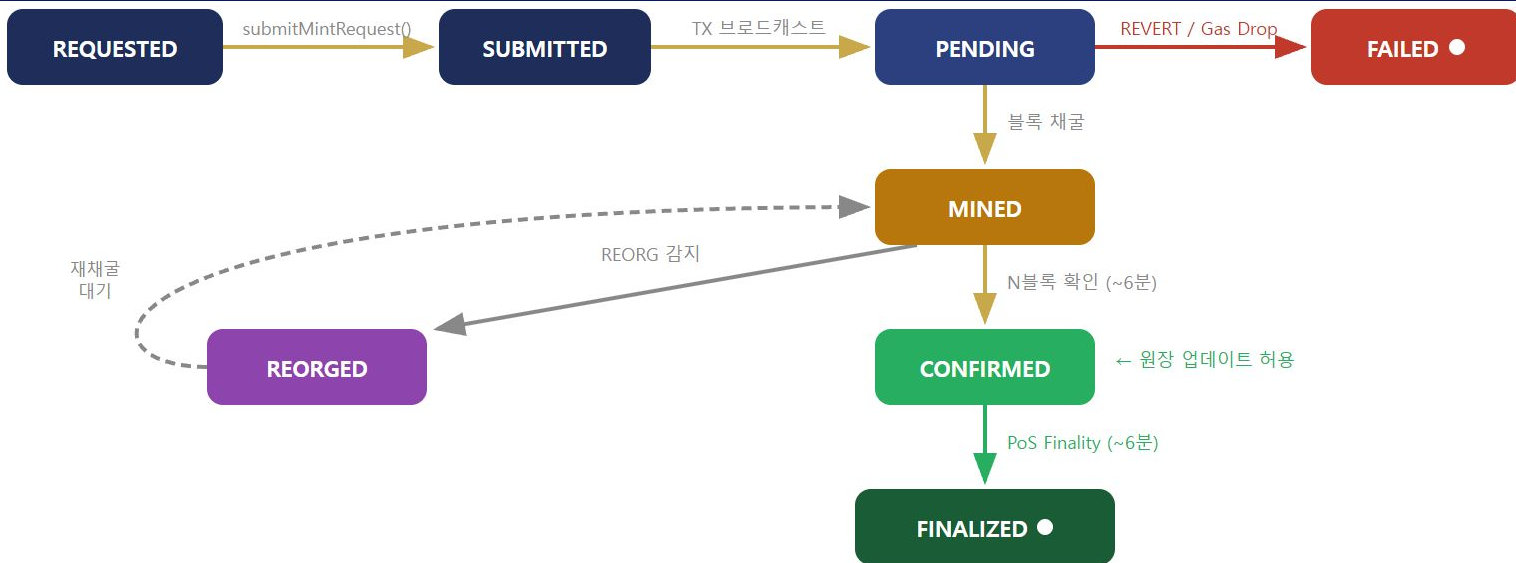
```
export type TxStatus =  
  'REQUESTED'    // 요청 생성, VASP 전송 전  
  | 'SUBMITTED'   // VASP에 전송됨, TX hash 미확득  
  | 'PENDING'     // TX 전송됨, 블록 미채굴  
  | 'MINED'       // 블록에 포함됨 (INCLUDED), Finality 미확보  
  | 'CONFIRMED'   // 충분한 블록 확인 → 원장 업데이트 허용  
  | 'FINALIZED'   // PoS 2/3+ validator 동의 → 절대 불변 — 종단  
  | 'FAILED'      // REVERT 또는 최종 실패 — 중단  
  | 'REORGED';    // MINED 후 REORG로 TX 소실, 재처리 대기
```

상태 정의의 원칙: 단순한 "단계"가 아니라 "이 상태에 있을 때 무엇이 참인가"를 기준으로 정의한다.

"PENDING이면 txHash가 있다" · "CONFIRMED면 원장 업데이트 가능" · "FINALIZED는 반복 불가" — 위반 시 버그

상태 전이도

STATE TRANSITION DIAGRAM



진행 중 채굴됨 확정됨 중단 실패 재처리

REORG는 MINED 구간에서만 처리. CONFIRMED 도달 시 충분한 블록이 쌓여 실질적 REORG 위험 없음 → CONFIRMED → REORGED 경로 없음. FINALIZED 이후에는 PoS 합의 규칙상 REORG 자체 불가.

REORGED vs FAILED 구분 기준 — “재시도하면 성공 가능한가?” REORGED: TX는 유효했으나 체인 재편으로 소실 → 재전송하면 성공 가능 | FAILED: REVERT(컨트랙트 거부) · 재시도 횟수 초과 → 같은 내용으로 재시도해도 실패 반복

VALID_TRANSITIONS 맵 구현

TRANSITION MAP · transitionStatus()

```
export const VALID_TRANSITIONS: Record<TxStatus, TxStatus[]> = {
  REQUESTED: ['SUBMITTED', 'FAILED'],
  SUBMITTED: ['PENDING', 'FAILED'],
  PENDING: ['MINED', 'FAILED'],
  MINED: ['CONFIRMED', 'REORGED', 'FAILED'], // REORG는 MINED 구간에서만
  CONFIRMED: ['FINALIZED'], // 충분한 블록 → PoS Finality 대기
  FINALIZED: [], // 종단 - PoS 절대 불변
  FAILED: [], // 종단 - 더 이상 전이 없음
  REORGED: ['MINED', 'FAILED'], // 재채굴 대기 or 포기
};

export function transitionStatus(current: TxStatus, next: TxStatus): void {
  const allowed = VALID_TRANSITIONS[current] ?? [];
  if (!allowed.includes(next)) {
    throw new InvalidStatusTransitionError(current, next);
  }
}
```

MintRequest 데이터 모델

DATA MODEL · 각 필드가 왜 존재하는가

MintRequest 인터페이스

```
export interface MintRequest {  
  id: string; // UUID - Idempotency key  
  userId: string;  
  tokenId: bigint;  
  amount: bigint;  
  status: TxStatus;  
  txHash?: string;  
  blockNumber?: number;  
  retryCount: number;  
  gasPriceGwei?: number;  
  failReason?: string;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

필드별 역할

필드	역할
id (UUID)	Idempotency key — VASP에도 동일 ID 전달, 중복 발행 방지
txHash	PENDING 이후 획득. TIMEOUT 시 gas bump 대상 식별
blockNumber	MINED 시 기록. REORG 감지 시 현재 블록과 비교 기준점
retryCount	gas bump 횟수 추적 — 무한 재시도 방지
gasPriceGwei	gas bump 이력 — 단계별 인상 금액 추적
failReason	FAILED 전이 시 원인 저장 — 운영·감사·CS 대응

submitMintRequest — REQUESTED → SUBMITTED

TxStateMachineService.ts:128

```
async submitMintRequest(params: { userId, tokenId, amount }): Promise<string> {
  const id = randomUUID(); // ㉠ Idempotency key 생성

  const req: MintRequest = {
    id, userId: params.userId, tokenId: params.tokenId, amount: params.amount,
    status: 'REQUESTED', // ㉡ 초기 상태
    retryCount: 0, createdAt: new Date(), updatedAt: new Date(),
  };

  await this.repo.save(req); // ㉢ DB INSERT 먼저 — 여기서 크래시나도 복구 가능

  try {
    const walletAddr = await this.wallet.getWalletAddr(params.userId);
    // Phase 1: vaspAdapter.submitTransaction() — 인가 VASP REST API
    // Phase 3+: chainAdapter.sendTransaction() (직접 Custody)
    const { txHash } = await this.vasp.submitMint({
      to: walletAddr, tokenId: params.tokenId, amount: params.amount,
      requestId: id, // ㉣ VASP에도 동일 ID 전달
    });
    await this.repo.updateStatus(id, 'SUBMITTED', { txHash }); // ㉤ SUBMITTED 전이
  } catch (err) {
    await this.repo.updateStatus(id, 'FAILED', {
      failReason: `submit failed: ${String(err)}` // ㉥ 실패 시 즉시 FAILED
    });
    throw err;
  }
}
```


핸들러 패턴 — 가드 + 단일 책임

AT-LEAST-ONCE GUARD · return vs throw

핸들러 구현 패턴

```
// Phase 1: 인가 VASP Webhook → Redis Streams → Consumer 경로
async handleMined(requestId, blockNumber): Promise<void> {
  const req = await this._getOrThrow(requestId);
  if (req.status !== 'PENDING' && req.status !== 'SUBMITTED') return; // ← 가드
  await this.repo.updateStatus(requestId, 'MINED', { blockNumber });
}

async handleConfirmed(requestId): Promise<void> {
  const req = await this._getOrThrow(requestId);
  if (req.status !== 'MINED') return; // ← 가드
  await this.repo.updateStatus(requestId, 'CONFIRMED');
}

async handleFailed(requestId, reason): Promise<void> {
  await this.repo.updateStatus(requestId, 'FAILED', { failReason: reason });
}
```

왜 throw가 아닌 return인가

- Webhook / Redis Consumer는 **At-least-once**
- 같은 이벤트가 네트워크 오류·재시작으로 두 번 이상 올 수 있음

throw 선택 시: 에러 로그 → 불필요한 알람 → 운영자 대응 낭비

At-least-once 환경에서 "정상적인 중복"을 에러로 취급

return 선택 시: 이미 완료된 상태 → 할 일 없음 → 멍등 (Idempotent) 처리 → 중복 호출이 와도 데이터 변화 없음 ✓

이미 CONFIRMED된 TX에 handleMined 재배달 → return → PEL 정상 XACK → 멍등 처리

Observer Pattern 구현과 활용

TxTransitionEvent · EventEmitter extends

TxTransitionEvent + 구독 방법

```
export interface TxTransitionEvent {
  requestId: string;
  from:      TxStatus;
  to:        TxStatus;
  req:       MintRequest; // 전이 시점 스냅샷
}

export class TxStateMachineService extends EventEmitter {
  private async _transition(req, to, extra?) {
    const from = req.status;
    await this.repo.updateStatus(req.id, to, extra);
    this.emit('transition', { requestId: req.id, from, to,
                             req: { ...req, status: to } });
  }
}

svc.on('transition', (evt: TxTransitionEvent) => {
  console.log(`[${evt.requestId}] ${evt.from} → ${evt.to}`);
});
```

실제 활용 — 3가지 구독자

```
// 1. 감사 로그
svc.on('transition', async (evt) => {
  await auditLog.append({ action: 'TX_STATE_CHANGE',
    requestId: evt.requestId, from: evt.from, to: evt.to });
});

// 2. 운영 메트릭
svc.on('transition', (evt) => {
  metrics.increment(`tx.transition.${evt.from}_to_${evt.to}`);
  if (evt.to === 'FAILED') metrics.increment('tx.failed.total');
});

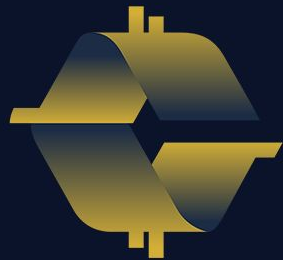
// 3. FAILED 알림
svc.on('transition', async (evt) => {
  if (evt.to === 'FAILED')
    await alertService.send({ severity: 'HIGH',
      message: `TX failed: ${evt.requestId}` });
});
```

3가지 구독자 모두 TxStateMachineService 코드 한 줄도 수정 안 함



chain-adapters

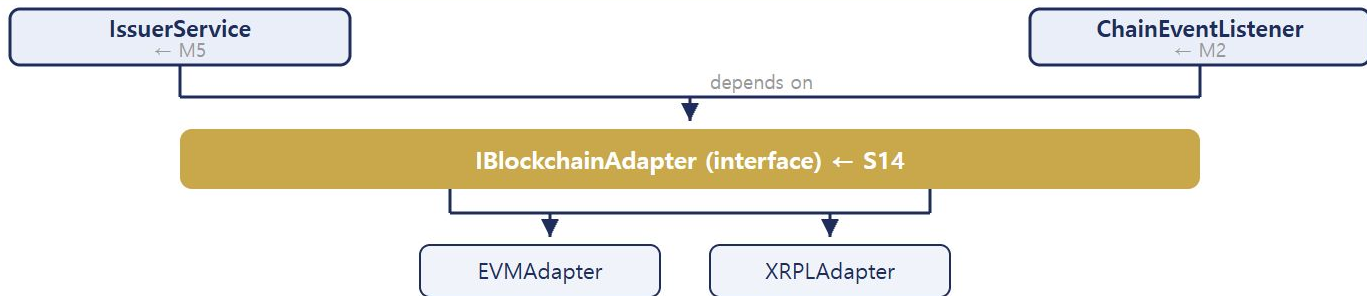
IBlockchainAdapter 인터페이스 설계 · 의존성 주입 경로 · 어댑터 교
체 전략



COINCRAFT
ACADEMY

S14 — IBlockchainAdapter 의존 구조

WHO DEPENDS ON IBlockchainAdapter — M2 · M3 · M5



인터페이스	설명	교체 지점	세션
IBlockchainAdapter	체인 연산 추상화 — sendTransaction-getReceipt-subscribeEvents	EVM→XRPL→Circle	S14
IVASPAAdapter	VASP API 추상화 — submitTransaction-getTransactionStatus	KyoboVASP→ExternalVASP	S21
VaspTxClient	TX 실행 위임 — submitMint-getStatus-resubmitWithGasBump	External→Custody	S13
ISignerService	서명 추상화 — sign(SignRequest). Phase 3 내재화 핵심 교체 지점	소프트웨어→HSM·KMS	Phase 3
ICoreBankingAdapter	코어뱅킹 추상화 — getUserAccount-syncBalance-sendRewardNotification	내부 API 변경	—
IKYCProvider	KYC 추상화 — getKYCStatus(userId) → KYCStatus	KYC 벤더 교체	—

IBlockchainAdapter 설계 원칙 2/3

STRATEGY PATTERN — BLOCKCHAIN ADAPTER

블록체인 어댑터도 동일하다 — 비즈니스 레이어는 IBlockchainAdapter 인터페이스만 안다. EVM인지 XRPL인지 Circle인지 모른다.

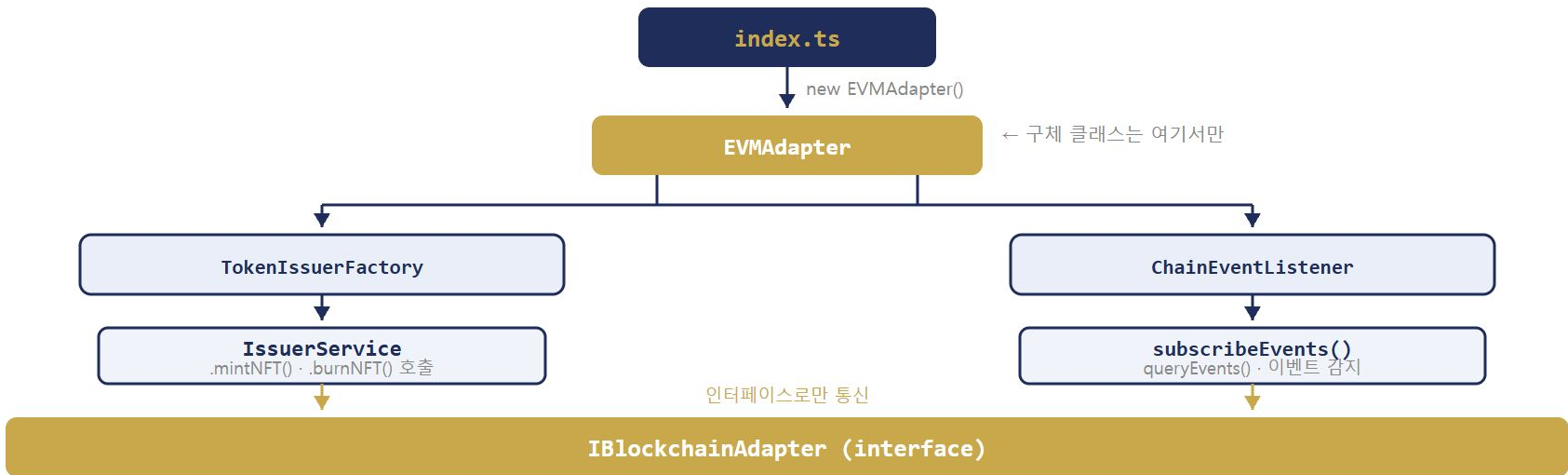


교체 원칙 — 어댑터 교체 = DI 설정 한 줄 변경. 비즈니스 코드는 변경 없음.

실제 운영 코드의 주입 경로 5/6

DI IN PRODUCTION — FULL FLOW SUMMARY

전체 흐름 정리



원칙 — 구체 클래스(EVMAAdapter)는 오직 index.ts 한 곳에서만 등장한다. 나머지 모든 코드는 IBlockchainAdapter 만 안다.

UUID as Idempotency Key

§ 2 requestId 설계 · M3 S16

requestId 발급 원칙

- **발급 시점:** 요청 생성 시 단 1회 — 재시도 시 재사용
- **발급 방법:** `crypto.randomUUID()` — RFC 4122 v4
- **전달 범위:** TxStateMachineService → VaspTxClient 전 레이어 관통
- **보관 위치:** DB `tx_requests.request_id` UNIQUE 컬럼
- **VASP 역할:** 동일 requestId 재수신 → 기존 txHash 반환 (신규 TX 생성 안 함)

VaspMockClient 중복 판별 로직

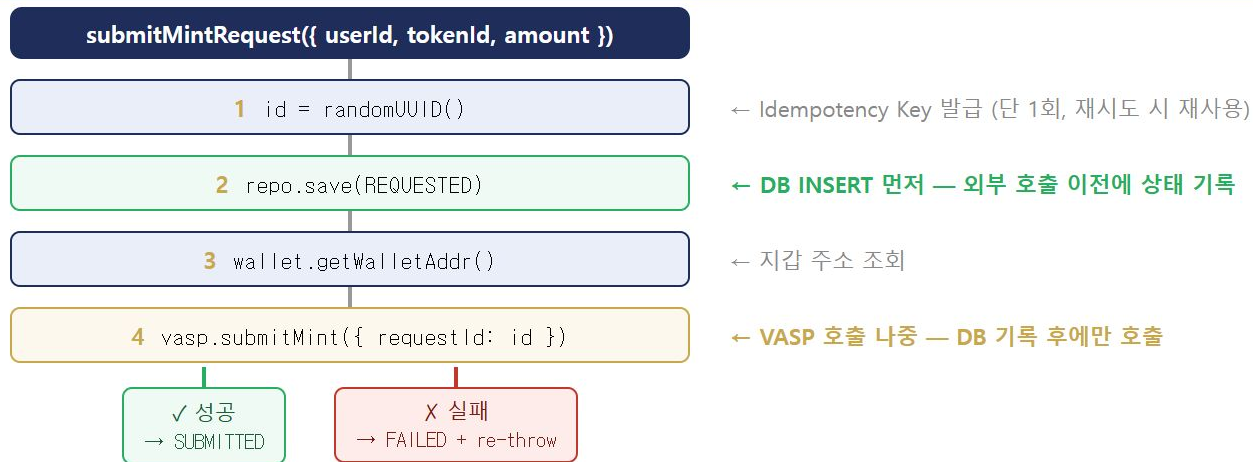
```
if (this.submitted.has(requestId)) {  
    // 이미 처리된 요청 → 캐시된 txHash 반환  
    return this.submitted.get(requestId);  
}  
  
// 신규 요청 → 새 txHash 생성 후 Map 등록  
const txHash = generateTxHash(requestId);  
this.submitted.set(requestId, txHash);  
return txHash;
```

같은 requestId 3번 → 항상 동일 txHash 반환

다른 requestId → 새 txHash 생성

submitMintRequest 흐름

§ 2 requestId 설계 · M3 S16



순서 불변 원칙: 2(DB) → 4(VASP) 순서는 바뀌지 않는다. DB 기록 없이 VASP를 먼저 호출하면 크래시 후 상태 추적 불가.

Exponential Backoff

§ 1 재시도 전략

공식: $\text{delay} = \min(\text{initialDelay} \times 2^{\text{attempt}}, \text{maxDelay})$

지수 증가 예시 (initialDelay = 1s, max = 30s)

- attempt 0 → 1s
- attempt 1 → 2s
- attempt 2 → 4s
- attempt 3 → 8s
- attempt 4 → 16s
- attempt 5 → 30s (cap)

코드

```
let delayMs = initialDelayMs;

for (let attempt = 1; attempt <= maxAttempts; attempt++) {
  try { return await fn(); }
  catch (err) {
    if (nonRetryable(err)) throw err;
    if (attempt === maxAttempts) throw err;

    const jittered = delayMs * (0.8 + Math.random() * 0.4);
    await sleep(jittered);
    delayMs = Math.min(delayMs * backoffFactor, maxDelayMs);
  }
}
```

의미: 실패할수록 더 오래 기다린다 → VASP가 복구할 시간을 확보한다

Jitter — Thundering Herd 방지

§ 1 재시도 전략

Thundering Herd: 여러 인스턴스가 동일한 Backoff 간격으로 동시에 재시도 → 결국 동시 집중

Jitter 적용 방법

- baseDelay에 $\pm 20\%$ 무작위 편차 추가
- 범위: $\text{baseDelay} \times 0.8 \sim \text{baseDelay} \times 1.2$
- 인스턴스마다 다른 시점에 재시도

```
const jittered =  
  delayMs * (0.8 + Math.random() * 0.4);  
await sleep(jittered);  
delayMs = Math.min(delayMs * backoffFactor, maxDelayMs);
```

효과 비교

Jitter 없음 (1,000 인스턴스):

t=2s → 1,000개 동시 재시도 ❌

Jitter 적용:

t=1.6s → 50개

t=1.8s → 200개

t=2.0s → 500개

t=2.2s → 200개 ← 분산됨

효과: 재시도 시점이 시간축으로 분산 → VASP 부하가 고르게 분산된다

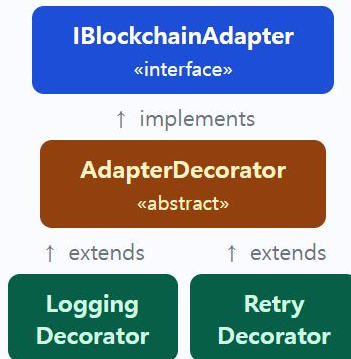
Decorator 3계층 구조

§ 2 Decorator 패턴

계층 역할

- **IBlockchainAdapter** — 인터페이스. 메서드 시그니처만 선언.
EVMAdapter-Decorator 모두 이걸 구현 → 호출자는 누가 들어와도 동일하게 사용
- **AdapterDecorator** — abstract 베이스. inner를 받아 모든 메서드를 `this.inner.xxx()`로 위임. 서브클래스는 바꿀 메서드만 `override`
- **Logging / Retry** — 구현체. extends AdapterDecorator. 필요한 메서드만 `override`해서 앞뒤로 코드 끼워넣음

구조도



베이스 클래스가 필요한 이유: Logging-Retry 모두 "10개 메서드를 inner에 위임"하는 동일한 코드가 필요 → 베이스 클래스가 한 번만 작성, 서브클래스는 override만

Circuit Breaker 3가지 상태

§ 3 Circuit Breaker

CLOSED — 정상

- 모든 요청 통과
- 실패 카운터 누적
- 임계치 미달 시 유지
- 연속 5회 실패 → OPEN

OPEN — 차단

- 모든 요청 즉시 거절
- CircuitOpenError 발생
- VASP에 요청 미전송
- 30s 후 → HALF_OPEN

HALF_OPEN — 탐색

- 탐색 요청 1개 허용
- 성공 → CLOSED 복귀
- 실패 → 다시 OPEN
- VASP 복구 여부 확인



목적: 장애 서비스에 대한 요청을 아예 차단해 복구 시간을 확보한다

3종 TX 장애 비교

§ 1 · 장애 유형별 발생 시점 · 원인 · 대응 전략

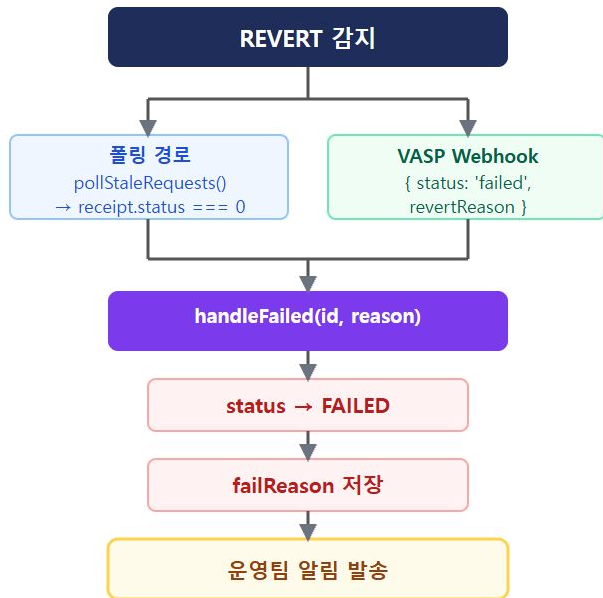
유형	발생 시점	원인	재시도?	대응
REVERT	블록 포함 후	컨트랙트 조건 미충족	❌ 금지	즉시 FAILED + reason 저장
TIMEOUT	mempool 대기 중	가스비 부족	✅ gas bump	PENDING 유지 → gas bump 후 재전송
REORG	블록 포함 후	체인 재편성	✅ 대기 후	REORGED → 5블록 대기 후 재확인

핵심 구분: 같은 FAILED처럼 보여도 원인이 다르면 대응이 달라진다 — REVERT는 원인 수정 없이 재시도 불가, TIMEOUT·REORG는 재시도 가능

REVERT 후 처리 정책

§ 1 · 즉시 FAILED 전이 + 자동 재시도 금지

처리 순서



재시도 정책

- 자동 재시도 금지 — 원인 수정 없이 재시도 = 가스비 낭비
- 운영 플로우 — 알림 → 원인 파악 → 수동 재발행

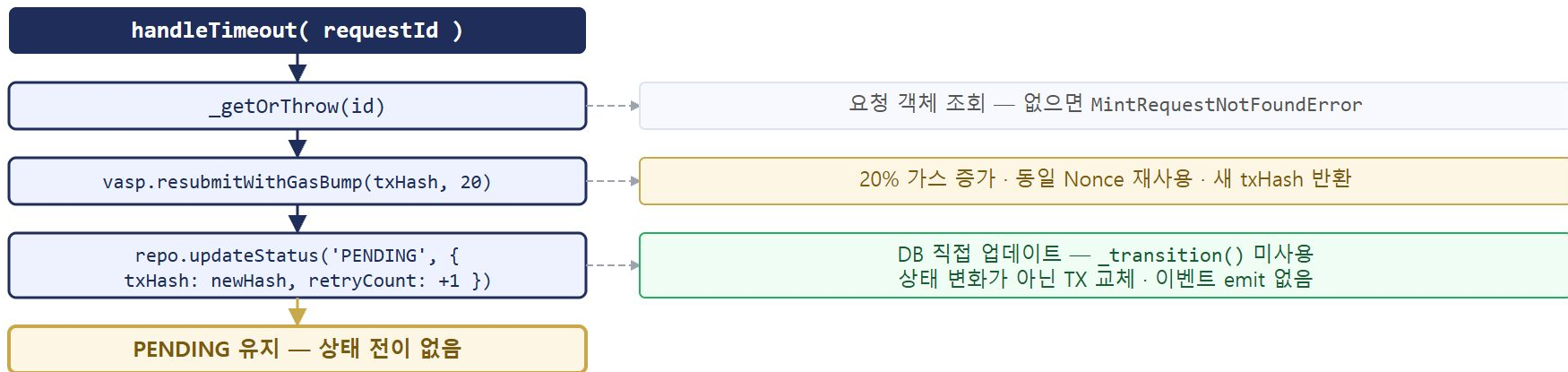
reason 분류별 해결 방향

분류	해결 방향
BALANCE	잔액 보충 후 재발행
PAUSED	컨트랙트 일시정지 해제
ACCESS	권한 부여 후 재발행

REVERT reason을 저장해야 운영팀이 원인을 파악할 수 있다 — reason 없이 FAILED만 기록하면 디버깅 불가

handleTimeout 흐름

§ 2 Gas Bump · 상태 전이 없는 TX 교체 패턴



`_transition()` 미사용 이유: PENDING→PENDING은 VALID_TRANSITIONS에 정의되지 않음. 상태 변화가 아니라 TX 교체가므로 `repo.updateStatus()` 직접 호출.

이벤트 emit 없음: 외부 원장에 반영할 상태 변화가 없다. txHash만 갱신.

handleReorg 흐름

§ 4 REORGED 전이 설계 · 대기 후 재확인 패턴



핵심: `reorgedReq` — `_transition()` 두 번째 호출 시 전달하는 객체는 DB에서 새로 조회한, `status`가 `REORGED` 인 객체여야 한다.

3종 복구 전략 비교

§ 5 복구 전략 정리 · 장애 유형별 감지 · 대응 · 최종 상태

장애 유형	감지 방법	즉각 대응	대기	최종 상태
REVERT	VASP webhook	즉시 FAILED + failReason 기록	없음	FAILED
TIMEOUT	폴링 (N분 이상 PENDING)	gas bump 20% 재전송	없음	PENDING → MINED
REORG	VASP webhook	REORGED 상태 전이	5블록 (~60s)	MINED 또는 FAILED

핵심 구분: 세 가지 모두 "TX가 처리 안 됐다"처럼 보이지만 원인과 대응이 완전히 다르다. 원인 진단 없이 재전송하면 이중 처리 위험이 생긴다.

왜 VASP를 인터페이스로 추상화하는가

VENDOR LOCK-IN 방지 — PHASE 전환 비용 최소화

문제 — 직접 호출 시

IssuerService → VASP REST API 직접 호출

Phase 3에서 교보 자체 VASP 취득 시:

- IssuerService 코드 **전체** 수정
- 테스트 교체, 배포 리스크 증가

규제 맥락 (특금법)

가상자산사업자(VASP) 신고·인가 없이 직접 운영 불가

→ Phase 1: 인가받은 외부 VASP에 TX 위탁

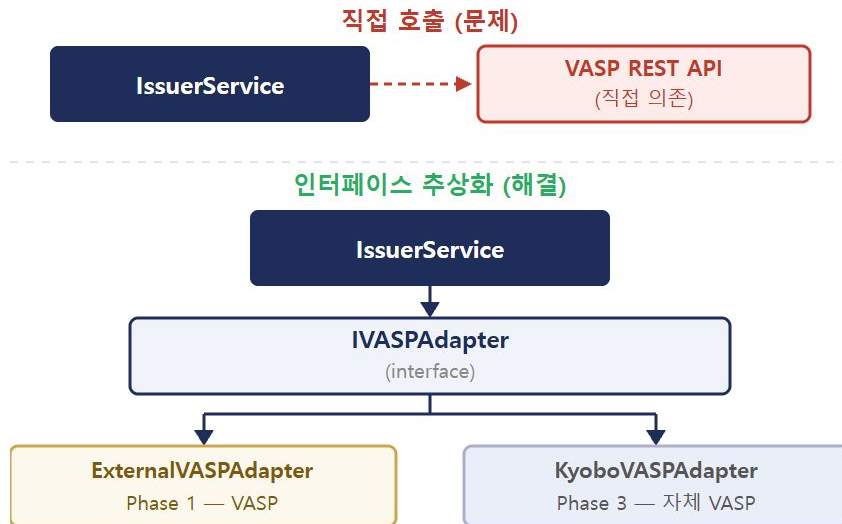
→ Phase 3: 교보생명 직접 VASP 인가 취득 시 내재화

해결 — DI 교체 1줄

IssuerService는 **IVASPAdapter** 만 알면 됨

Phase 전환 = DI 바인딩 교체만으로 완료

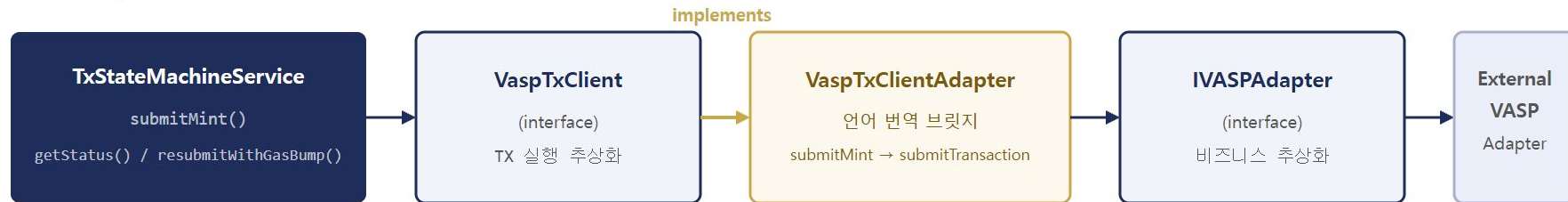
IssuerService 코드 수정 없음



VaspTxClientAdapter (1/2) — 왜 필요한가

BRIDGE LAYER — INTERFACE LANGUAGE TRANSLATION

internal/packages/vasp/src/tx/VaspTxClientAdapter.ts



VaspTxClient — TX 실행 인터페이스

메서드	역할
submitMint()	NFT 발행 TX 제출
getStatus()	TX 상태 조회
resubmitWithGasBump()	gas 인상 재전송 (Phase 3)

TxStateMachineService가 요구하는 언어 — TX 실행에 특화된 낮은 레벨 인터페이스

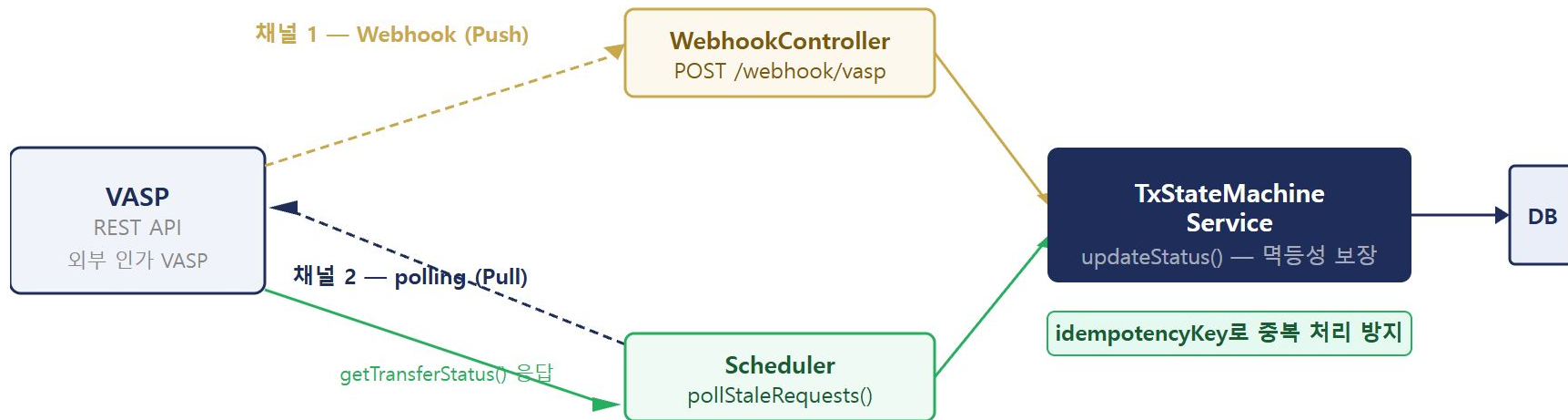
IVASPAdapter — 비즈니스 인터페이스

메서드	역할
submitTransaction()	TX 위탁 (일반 용어)
getTransferStatus()	이체 상태 조회
createWallet / transfer	지갑 관리 / 전송

비즈니스 언어 — 더 높은 레벨 — VASP 서비스 전체를 표현. SRP: 하나의 역할 = 하나의 인터페이스

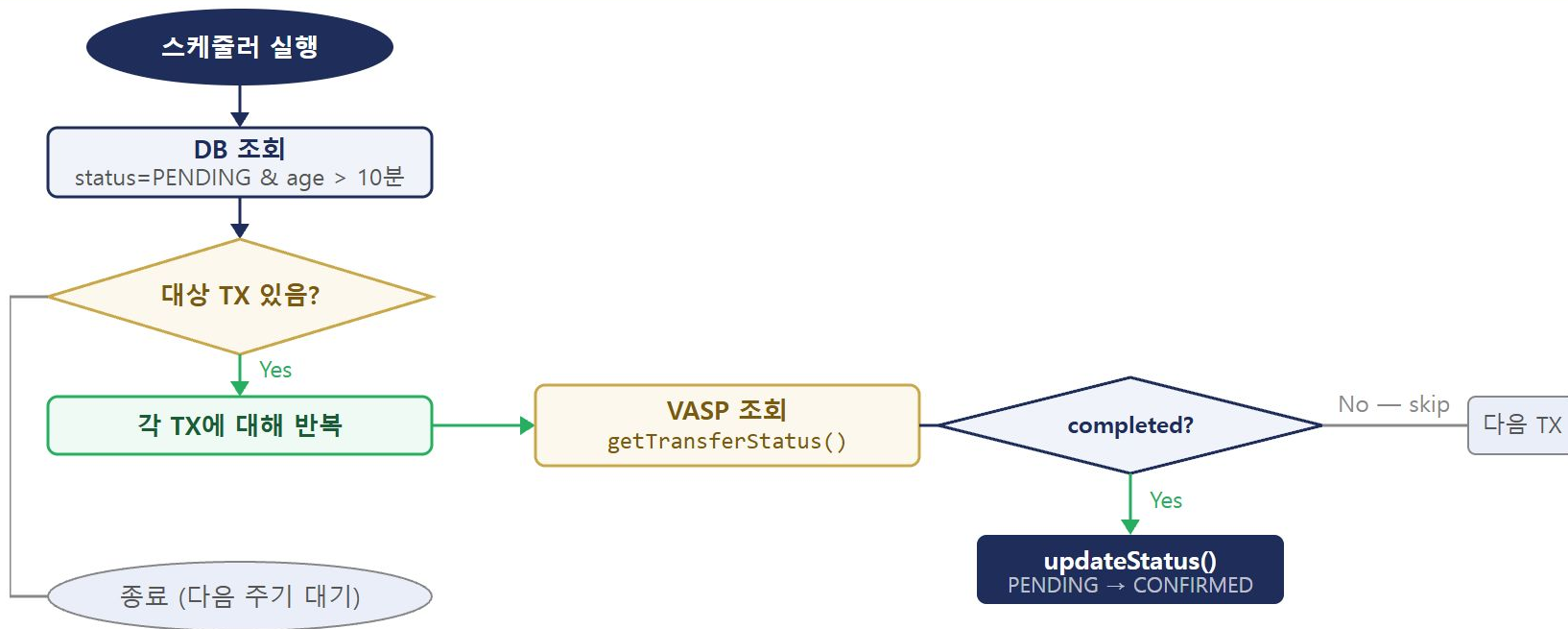
전체 아키텍처 — 이중 채널 전체 그림

DUAL CHANNEL ARCHITECTURE



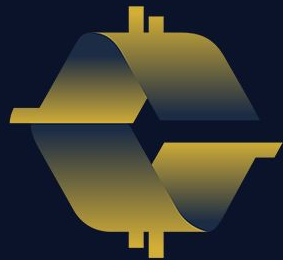
pollStaleRequests 알고리즘 설계 (1/2)

POLLING ALGORITHM — FLOWCHART



IV core-banking

LedgerService · MintStatus · ReconcileService · AuditLogService ·
Java 내부원장 연동



COINCRAFT
ACADEMY

4개 테이블 역할 분담

각 테이블이 무엇을 담당하는가 — 단일 역할 원칙 (Single Responsibility)

오프체인 원장 (LedgerService)

mint_requests

TX in-flight 추적 · 30일 만료

processed_events

이벤트 중복 방지 · 90일 보존

user_nft_holdings

현재 보유 캐시 · Java 영구 원장

audit_log

변경 불가 감사 기록 · 5년 보존

테이블	역할	관리 주체	데이터 수명
mint_requests	TX in-flight 상태 추적 (REQUESTED→FINALIZED)	Node.js LedgerService	CONFIRMED/FAILED 후 30일
processed_events	NFTIssued 이벤트 최초 1회만 처리 보장	Node.js LedgerService	90일 보존 (Reorg 감지)
user_nft_holdings	현재 NFT 보유 현황 캐시	Java Gateway 영구 원장	영구 (법적 의무)
audit_log	모든 상태 변경 불변 기록 · SHA-256 체크섬	Java Gateway + AuditLogService	5년 (가상자산법)

중요: mint_requests / processed_events는 Node.js 서비스의 임시적·기술적 데이터.

user_nft_holdings / audit_log는 Java Gateway가 관리하는 영구 금융 원장. S23~S24는 Node.js 레이어만 다룬다.

① mint_requests — TX in-flight 상태 추적

발행 요청 생성 → TX 제출 → 블록 포함 → 확정까지 전 과정 상태 기록 · 30일 만료

```
-- @ mint_requests
```

```
CREATE TABLE mint_requests (  
  id          UUID          PRIMARY KEY,  
  user_id     VARCHAR(64)   NOT NULL,  
  policy_id   VARCHAR(64)   NOT NULL,  
  status      VARCHAR(16)   NOT NULL DEFAULT 'REQUESTED',  
  tx_hash     VARCHAR(66),   -- SUBMITTED 이후  
  token_id    NUMERIC,       -- CONFIRMED 이후  
  block_number NUMERIC,      -- MINED 이후  
  error_msg   TEXT,         -- FAILED 시  
  retry_count INTEGER NOT NULL DEFAULT 0,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_mint_requests_user  
  ON mint_requests(user_id);  
CREATE INDEX idx_mint_requests_status  
  ON mint_requests(status); -- pollStaleRequests용
```

실제 레코드 예시 (MINED 상태 — token_id 아직 NULL)

```
request_id : "a1b2c3d4-..."  
user_id    : "K-20240001"  
policy_id  : "WALK-10000"  
status     : "MINED"  
tx_hash    : "0x3f2a...8e91"  
token_id   : NULL      ← 아직 확정 전이라 모름  
error_msg  : NULL  
created_at : 2024-03-15 09:01:00  
updated_at : 2024-03-15 09:01:45 ← SUBMITTED 때 갱신
```

token_id가 NULL인 이유: tokenId는 온체인 이벤트(NFTIssued)를 수신해야 확정.

MINED = 블록 포함 확인이지만, 어떤 tokenId가 발급됐는지는 아직 모름.
ConsumerGroupWorker가 NFTIssued 수신 후 CONFIRMED 전이 시 채운다.

데이터 수명: CONFIRMED 또는 FAILED 확정 후 30일 자동 만료.
개인정보보호법 §21 — 처리 목적 달성 후 삭제 의무.

② processed_events — 이벤트 중복 방지 장치

at-least-once delivery에서 같은 이벤트를 단 1회만 처리 · UNIQUE(tx_hash, log_index)가 핵심

```
-- @ processed_events
CREATE TABLE processed_events (
  id          SERIAL          PRIMARY KEY,
  tx_hash     VARCHAR(66) NOT NULL,
  log_index   INTEGER         NOT NULL,
  event_name  VARCHAR(64)  NOT NULL,
  block_number NUMERIC        NOT NULL,
  payload     JSONB,
  processed_at TIMESTAMPTZ   DEFAULT NOW(),
  UNIQUE (tx_hash, log_index) -- 이 줄이 전부
);
CREATE INDEX idx_processed_events_block
ON processed_events(block_number);
```

왜 tx_hash만으로 안 되나?

TX 0x3f2a 안에: log[0]=NFTIssued(tokenId=1001, 0xAAA) ·

log[1]=NFTIssued(tokenId=1002, 0xBBB)

tx_hash만 UNIQUE면 log[1]이 "이미 처리됨"으로 잘못 차단 → 0xBBB는 NFT 못 받음.

log_index가 한 TX 안에서 각 이벤트를 구분하는 유일한 키.

실제 레코드 예시

```
id          : 1
tx_hash     : "0x3f2a...8e91"
log_index   : 0
event_name  : "NFTIssued"
block_number : 12345
payload     : {"tokenId": 1001, "to": "0xAAA..."}
processed_at : 2024-03-15 09:01:50
```

2번째

같은 (tx_hash=0x3f2a, log_index=0)로 INSERT 시도
→ ON CONFLICT DO NOTHING → rows.length==0
→ { skipped: true } → 상위 호출부 즉시 return



DB UNIQUE 제약이 단일 원자 연산으로 처리
Race Condition 없음 — SELECT-then-INSERT보다 안전

데이터 수명: 90일 보존. Reorg 감지를 위해 최근 블록 이벤트 유지 필요.

③ user_nft_holdings — 현재 NFT 보유 현황 캐시

앱 "내 NFT" 목록의 실제 데이터 소스 · 온체인 RPC 없이 5ms 응답 · Java Gateway 영구 원장

```
-- © user_nft_holdings (Phase 1 확정)
CREATE TABLE user_nft_holdings (
  id          BIGSERIAL PRIMARY KEY,
  user_id     VARCHAR(64) NOT NULL,
  token_id    BIGINT     NOT NULL,
  contract_addr VARCHAR(42) NOT NULL,
  chain_id    INT        NOT NULL,
  amount      BIGINT     NOT NULL DEFAULT 1,
  acquired_at TIMESTAMPTZ NOT NULL,
  released_at TIMESTAMPTZ,
  on_chain_tx  VARCHAR(66) NOT NULL,
  UNIQUE(user_id, token_id, contract_addr, chain_id)
);
CREATE INDEX idx_holdings_user
  ON user_nft_holdings(user_id);
```

읽기 경로 (온체인 호출 없음):

```
SELECT * FROM user_nft_holdings WHERE user_id='K-20240001'
```

→ 인덱스 스캔 → 5ms vs. 온체인 balanceOf() RPC → 200~500ms

데이터 수명: 영구 보존 (법적 의무).

Java Gateway 영구 원장. ReconcileService(S25)가 주기적으로 온체인과 비교.

실제 레코드 예시 (발행 확정 후 삽입)

```
id          : 1
user_id     : "K-20240001"
token_id    : 1001
contract_addr : "0x4b0897b0..."
chain_id    : 1
amount      : 3
acquired_at  : 2024-03-15 09:01:50
on_chain_tx  : "0xaaaaa0000..."
released_at  : null
```

UNIQUE(user_id, token_id, contract_addr, chain_id)의 두 가지 역할:

- ① 비즈니스 규칙 — 같은 사람이 같은 (컨트랙트.tokenId)를 중복 보유 불가
- ② at-least-once 방어 — 재처리 시에도 중복 INSERT 자동 차단
mint_requests/processed_events가 뚫려도 이 제약이 최종 방어선.

쓰기: on-chain NFTIssued 이벤트 수신 후 Java Gateway 호출

```
INSERT ... ON CONFLICT (user_id, token_id, contract_addr, chain_id) DO
  NOTHING
```


④ audit_log — 변경 불가 감사 기록

전자금융감독규정 §34 (접근기록 1년+) · 가상자산이용자보호법 §15 (거래기록 5년) · INSERT-only

```
-- ④ audit_log - INSERT-only
CREATE TABLE audit_log (
  id          BIGSERIAL    PRIMARY KEY,
  event_time  TIMESTAMPTZ  NOT NULL,
  actor       VARCHAR(64)  NOT NULL,
  action      VARCHAR(64)  NOT NULL,
  resource_type VARCHAR(32) NOT NULL,
  resource_id VARCHAR(128) NOT NULL,
  before_state JSONB,
  after_state JSONB        NOT NULL,
  prev_checksum VARCHAR(64), -- NULL = genesis
  checksum     VARCHAR(64) NOT NULL
  -- SHA-256(prevChecksum+eventTime+actor+action+resourceId+afterState)
);
CREATE INDEX idx_audit_resource
  ON audit_log(resource_type, resource_id);
CREATE INDEX idx_audit_actor ON audit_log(actor);
```

실제 레코드 예시 (발행 요청 생성 시점)

```
actor       : "system:IssuerService"
action      : "MINT_REQUESTED"
resource_type : "mint_request"
resource_id  : "a1b2c3d4-..."
before_state : NULL
after_state  : {"status":"REQUESTED","userId":"K-20240001"}
prev_checksum : "0000...0"
checksum     : "e3b0c44298fc..."
```

before_state: 최초 생성 시 NULL · 상태 변경 시 변경 전 스냅샷

INSERT-only 원칙: 레코드 수정·삭제 절대 금지. 보정은 새 레코드 추가.
위반 시 전자금융감독규정 §34 · 가상자산이용자보호법 §15 위반.

Hash chain checksum:

```
SHA-256(prevChecksum + eventTime + actor + action + resourceId +
afterState)
```

수정 후 checksum 재계산해도 다음 레코드 prevChecksum과 불일치 → 탐지
첫 레코드 prevChecksum = "0000...0" (genesis) · S26에서 상세히 다룬다.

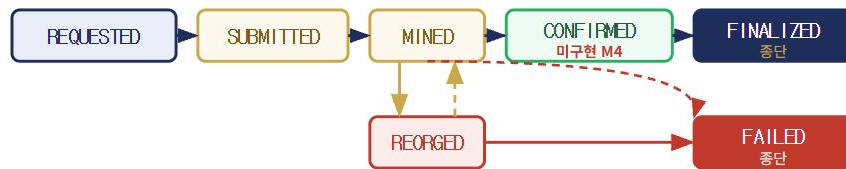
MintStatus 타입 7종 + VALID_TRANSITIONS 맵

LedgerService가 관리하는 상태 전이 — 허용된 경로만 통과

```
export type MintStatus =  
  | 'REQUESTED' // 발행 요청 생성 (초기 상태)  
  | 'SUBMITTED' // VASP에 TX 제출 완료  
  | 'MINED' // 블록 포함 — Reorg 가능  
  | 'CONFIRMED' // 내부 원장 반영 완료 (종단)  
  | 'FINALIZED' // PoS Finality 확보 (종단)  
  | 'FAILED' // 발행 실패 (종단)  
  | 'REORGED'; // 체인 재편으로 TX 소실
```

```
private static readonly VALID_TRANSITIONS:  
  Record<MintStatus, MintStatus[]> = {  
    REQUESTED: ['SUBMITTED', 'FAILED'],  
    SUBMITTED: ['MINED', 'FAILED'],  
    MINED: ['CONFIRMED', 'REORGED', 'FAILED'],  
    CONFIRMED: ['FINALIZED'],  
    FINALIZED: [], // 종단 — 원장 업데이트 완료  
    FAILED: [], // 종단  
    REORGED: ['MINED', 'FAILED'],  
  };
```

상태 전이 다이어그램:



* REQUESTED · SUBMITTED → FAILED 직접 전이 가능

상태	txHash	설정 주체
REQUESTED	NULL	LedgerService.createMintRequest()
SUBMITTED	있음 (필수)	TxStateMachineService
MINED	있음	TxStateMachineService
CONFIRMED	있음 (필수)	ConsumerGroupWorker 미구현 — M4
FINALIZED	있음	TxStateMachineService
REORGED	있음 (무효)	TxStateMachineService
FAILED	있을 수도	TxStateMachineService

MintRequest 인터페이스 + createMintRequest() 구현

요청 생성 시 무엇이 INSERT되고 AuditLog에 무엇이 기록되는가

MintRequest 인터페이스 필드

필드	타입	설명
id	string	UUID — randomUUID() 생성
userId / policyId	string	요청자 · 정책 식별자
status	MintStatus	현재 상태 (7종)
txHash?	string	SUBMITTED 이후 설정
tokenId?	bigint	CONFIRMED 이후 설정
errorMsg?	string	FAILED 시 원인 메시지

bigint 이유: EVM tokenId는 uint256 (최대 $2^{256}-1$). JS `number` 는 $2^{53}-1$ 초과 시 정밀도 손실 → **bigint** 필수.

createMintRequest() 처리 흐름

- 1 **UUID 생성** — `randomUUID()` (Node.js crypto 모듈). 호출자가 아닌 서비스가 직접 생성.
- 2 **DB INSERT** — `mint_requests` 에 `status= 'REQUESTED'` 으로 삽입. `created_at` 과 `updated_at` 에 동일 타임스탬프(\$4,\$4).
- 3 **감사 기록** — `auditLog.log()` 호출. 최초 생성이므로 `beforeState` 없음, `afterState` 만 기록.
- 4 **객체 반환** — DB 재조회 없이 INSERT에 사용한 값으로 바로 반환. DB 왕복 1회 절감.

auditLog.log() 패턴: DB INSERT와 감사 기록을 항상 함께 실행. 어느 하나라도 빠지면 규제 위반.

updateMintRequest() — 상태 전이 가드 구현

허용되지 않은 전이를 DB 업데이트 전에 차단 · beforeState/afterState 감사 기록

처리 4단계

- 1 **현재 상태 조회** — `getMintRequest(requestId)` 호출. 없으면 `MintRequestNotFoundError` throw.
- 2 **전이 허용 체크** — `VALID_TRANSITIONS[current.status]` 에서 허용 목록 조회 → `includes(patch.status)` false면 `InvalidStateTransitionError` throw.
- 3 **DB UPDATE** — `status, tx_hash, token_id, error_msg, updated_at` 갱신. `tokenId`는 `bigint` → `string` 변환 후 저장.
- 4 **감사 기록** — `beforeState: current` (DB 조회값) · `afterState: {...current, ...patch}` . 변경 전/후 모두 기록.

설계 포인트

VALID_TRANSITIONS이 규칙을 소유

- 상태 전이 허용 여부는 코드 로직이 아닌 맵이 결정
- 새 상태 추가 시 맵만 수정 — 함수 로직은 불변
- 테스트 시 맵을 직접 검증 가능

beforeState를 DB 조회로 확보

- Step 1에서 조회한 `current`를 `beforeState`로 그대로 사용
- 호출자가 이전 상태를 넘겨줄 필요 없음 — 원장이 직접 확인

FAILED → CONFIRMED 시도:

`VALID_TRANSITIONS[FAILED] = []` → `includes('CONFIRMED') = false`
→ `InvalidStateTransitionError` throw

두 상태머신의 관계 — TxStateMachineService vs LedgerService

M3 외부 상태 동기화 + M4 내부 원장 일관성 — 독립적이지만 직렬로 작동

구분	TxStateMachineService (M3)	LedgerService (M4)
추적 대상	VASP/블록체인의 TX 상태	내부 발행 요청의 원장 상태
상태 타입	TxStatus (8개, PENDING-REORGED 포함)	MintStatus (7개)
위치	내부망 (VASP 통신 레이어)	Core-Banking (원장 레이어)
트리거	VASP 콜백, 블록 이벤트, 폴링	TxStateMachineService 호출
역할	외부 상태 동기화	내부 원장 일관성 보장
실패 시	재시도 (gas bump, REORG 복구)	InvalidStateTransitionError 예외
VALID_TRANSITIONS 필요 이유	블록체인 이벤트 순서 보장 안 됨	Consumer Worker 중복·순서 역전

M3 없이 M4가 혼자 동작하지 않는다: M3 상태머신이 먼저 외부(체인) 상태를 동기화하고, 그 결과가 M4 원장 상태 전이를 트리거한다.

updateMintRequest() — 3단계 가드 전체 구현

레코드 존재 확인 → 상태 전이 가드 → 필드 유효성 가드 → DB UPDATE → Audit log

```
async updateMintRequest(
  requestId: string,
  patch: { status: MintStatus; txHash?: string;
          tokenId?: bigint; errorMsg?: string },
): Promise<MintRequest> {
  // 1. 레코드 존재 확인
  const current = await this.getMintRequest(requestId);
  if (!current) throw new MintRequestNotFoundError(requestId);

  // 2. 상태 전이 가드
  const allowed = LedgerService.VALID_TRANSITIONS[current.status];
  if (!allowed.includes(patch.status))
    throw new InvalidStateTransitionError(current.status, patch.status);

  // 3. 필드 유효성 가드
  if (patch.status === 'SUBMITTED' && !patch.txHash)
    throw new Error('txHash is required for SUBMITTED');
  if (patch.status === 'CONFIRMED' && !patch.tokenId)
    throw new Error('tokenId is required for CONFIRMED');
```

```
// 4. DB UPDATE - COALESCE로 기존 값 보존
await this.db.query(
  `UPDATE mint_requests
   SET status      = $1,
       tx_hash     = COALESCE($2, tx_hash),
       token_id    = COALESCE($3, token_id),
       error_msg   = $4,
       updated_at  = NOW()
   WHERE id = $6`,
  [patch.status,
   patch.txHash ?? null,
   patch.tokenId?.toString() ?? null,
   patch.errorMsg ?? null,
   requestId],
);

// 5. Audit log - 상태 변경 기록
const afterState = { ...current, ...patch };
await this.auditLog.log({
  actor: 'system', action: `STATUS_${patch.status}`,
  resourceType: 'MintRequest', resourceId: requestId,
  beforeState: current, afterState,
});
return afterState as MintRequest;
}
```

먹등 이벤트 처리 — 중복 이벤트 도착 시나리오

Redis Streams at-least-once delivery → 같은 이벤트가 두 번 도착하는 시나리오

중복 이벤트 발생 시나리오

- 1 t=0: **Worker-A**가 NFTIssued 이벤트 처리 시작
- 2 t=1: mint_requests UPDATE (CONFIRMED) ✓
- 3 t=2: user_nft_holdings INSERT ✓
- ! t=3: **Worker-A가 XACK 전에 프로세스 종료** ← 장애!
- 4 t=4: Redis — XACK 없음 → PEL에 메시지 남음
- 5 t=5: **Worker-B**가 PEL 재처리 → 같은 이벤트 다시 처리 시작

가드만 있는 경우의 처리

- 6 t=6: mint_requests UPDATE (CONFIRMED) 시도 → 이미 CONFIRMED인 상태
- ! VALID_TRANSITIONS['CONFIRMED'] = [] → InvalidStateTransitionError 발생!
- 7 Worker-B가 에러 → DLQ로 이동? → 알림 발생 → 담당자 혼란

더 나은 방법: 상태 전이 가드가 막아주는 것은 하지만, 이미 처리된 이벤트를 먼저 판별하는 게 더 깔끔하다. 이게 바로 **M2에서 배운 먹등성 가드** —

`processed_events` 테이블에 이벤트 ID를 기록해 중복 수신 시 조용히 skip.

recordProcessedEvent 구현 — ON CONFLICT DO NOTHING

UNIQUE(tx_hash, log_index) + RETURNING id → rows.length로 신규/중복 판별

```
async recordProcessedEvent(
  txHash: string,
  logIndex: number,
  eventName: string,
  blockNumber: bigint,
  payload: unknown,
): Promise<ProcessedEventResult> {
  const result = await this.db.query(
    `INSERT INTO processed_events
      (tx_hash, log_index, event_name,
       block_number, payload)
    VALUES ($1, $2, $3, $4, $5)
    ON CONFLICT (tx_hash, log_index) DO NOTHING
    RETURNING id`,
    [txHash, logIndex, eventName,
     blockNumber.toString(), JSON.stringify(payload)],
  );
  if (result.rows.length === 0) {
    return { skipped: true }; // UNIQUE 충돌 - 이미 처리됨
  }
  return { skipped: false, id: result.rows[0].id as number };
}
```

ON CONFLICT DO NOTHING 동작:

UNIQUE(tx_hash, log_index) 충돌 시 → 아무것도 INSERT하지 않음 → RETURNING에서 행 반환 없음 → rows.length === 0 → skipped: true

같은 txHash, 다른 logIndex:

하나의 TX에 여러 이벤트 로그가 있을 수 있음. (txHash, logIndex) 쌍이 이벤트의 유일 식별자. 같은 txHash라도 logIndex가 다르면 별개 이벤트 → 각각

skipped: false

SELECT-then-INSERT는 안전하지 않다: 동시에 두 Worker가 SELECT → 둘 다 "없음"으로 판단 → 둘 다 INSERT 시도 → 중복 발행. ON CONFLICT가 원자적으로 처리.

IReconcileService — 전체 인터페이스

RECONCILESERVICE.TS — 탐지 전용 설계 원칙

메서드	Phase	역할
<code>reconcile()</code>	1	KRW 총발행량 vs 수탁 잔액 대조 → warn / critical 분류
<code>getLastResult()</code>	1	최근 대조 결과 인메모리 캐싱 (재실행 없이 조회)
<code>reconcileNftHoldings()</code>	1	단일 사용자 NFT 원장 ↔ 온체인 대조
<code>reconcileAllNftHoldings()</code>	1	전 사용자 일괄 배치 대조
<code>onMintEvent()</code>	2	Mint 이벤트 수신 → 증분 대조 트리거
<code>onBurnEvent()</code>	2	Burn 이벤트 수신 → 증분 대조 트리거

설계 원칙 — `mintBatch` · `forceMint` · `sendTransaction` 없음. `ReconcileService`는 탐지만 한다. 보정은 운영자가 별도 채널(DLQ 재처리·이벤트 재발행)로 수행.

ReconcileService 생성자 — 의존성 주입

4가지 의존성: COREBANKING + ONCHAIN + LEDGER + ALERTER

의존성	메서드	역할
coreBanking ICoreBankingAdapter	<code>getUserAccount(userId)</code>	사용자 지갑 주소(walletAddr) 조회 — NFT 대조 시 온체인 주소 확인
	<code>recordTransaction(tx)</code>	KRW_MINT · KRW_BURN 이벤트를 감사 원장에 기록
	<code>getTotalSupply()</code>	KRW 스테이블코인 온체인 총발행량 조회
onchain 블록체인 RPC	<code>getCustodyAccountBalance()</code>	교보생명 원화 수탁 계좌 잔액 조회
	<code>balanceOf(addr, tokenId)</code> <code>getNftHoldings(addr)</code>	ERC-1155 NFT 보유 수량 및 보유 목록 조회
	<code>getHoldings(userId)</code>	원장 기준 사용자 NFT 보유 tokenId 목록 조회
ledger 내부 원장 DB	<code>getAllUserIds()</code>	전체 사용자 목록 — <code>reconcileAllNftHoldings()</code> 배치 대조용
alerter 알림 시스템	<code>fire(message, severity)</code>	불일치 감지 시 warn / critical 알림 발송 (PagerDuty · Slack 등)

reconcileNftHoldings() — 핵심 구현

원장 holdings vs 온체인 balanceOf — 사용자별 비교

- 1 ledger.getHoldings(userId) — 원장 NFT 보유 tokenId 목록 조회
- 2 coreBanking.getUserAccount(userId) — 온체인 지갑 주소(walletAddr) 조회
- 3 원장 각 tokenId → onchain.balanceOf(addr, tokenId) → 0이면 **LEDGER_ONLY**
- 4 onchain.getNftHoldings(addr) → 원장에 없는 tokenId → **ONCHAIN_ONLY**
- 5 불일치 ≥ 10건 → **critical** / 1~9건 → **warn** / 0건 → isHealthy: true

순차 루프 이유 — balanceOf() RPC 호출마다 네트워크 왕복. Rate Limit 초과 방지를 위해 순차 처리. 실제 운영에서는 batchSize로 청크 처리.

generateChecksum() — Node.js crypto 구현

AUDITLOGSERVICE.TS — createHash('sha256') 사용법

```
import { createHash } from 'crypto';

private generateChecksum(
  eventTime: Date, actor: string, action: string,
  resourceId: string, afterState: unknown,
): string {
  const raw = eventTime.toISOString() + actor + action
    + resourceId + JSON.stringify(afterState);
  return createHash('sha256').update(raw, 'utf8').digest('hex');
}
```

구분자 없음의 함정

"A"+"BC" 와 "AB"+"C" 는 raw가 동일 → 필드 경계 충돌 가능. 실운영에서는 이벤트 타입 조합이 고정돼 실용적으로 무해하나, 엄격하게는 구분자 추가 고려.

Date vs string 주의

파라미터는 Date 객체로 받아 내부에서 toISOString() 호출. string으로 받으면 포맷 불일치 → verifyIntegrity() 재계산 시 불일치 발생.

DatabaseClient 인터페이스 — raw SQL 추상화

ORM 없이 직접 쿼리 — 감사 로그의 특수성

```
// AuditLogService 내부에서 사용하는 DB 클라이언트 인터페이스
interface DatabaseClient {
    query(sql: string, params?: unknown[]): Promise<{ rows: Record<string, unknown>[] }>;
}

// 생성자 - DatabaseClient 주입
export class AuditLogService {
    constructor(private readonly db: DatabaseClient) {}
}
```

왜 ORM을 쓰지 않나?

- ORM이 UPDATE 쿼리를 자동 생성할 수 있음
- Append-only 강제를 ORM 레벨에서 보장 어려움
- raw SQL로 의도 명확히 표현
- 감사 로그는 ORM 추상화가 오히려 위험

테스트용 In-Memory Mock

- 실제 PostgreSQL 없이 테스트 가능
- INSERT 쿼리 파싱 → rows[] 배열에 추가
- SELECT WHERE id = ? → rows 필터
- checksum 조작 테스트 가능

log() — 전체 구현

감사 로그 INSERT + checksum 자동 생성

```
async log(params: LogParams): Promise<number> {
  const eventTime = new Date();
  const checksum = this.generateChecksum(
    eventTime, params.actor, params.action, params.resourceId, params.afterState,
  );
  const { rows } = await this.db.query(
    `INSERT INTO audit_log
      (event_time, actor, action, resource_type, resource_id,
       before_state, after_state, ip_address, session_id, checksum)
    VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10) RETURNING id`,
    [ eventTime.toISOString(),
      params.actor, params.action, params.resourceType, params.resourceId,
      params.beforeState !== undefined ? JSON.stringify(params.beforeState) : null,
      JSON.stringify(params.afterState),
      params.ipAddress ?? null, params.sessionId ?? null, checksum ],
  );
  return rows[0]!['id'] as number;
}
```

verifyIntegrity() — 단일 레코드 무결성 검증

저장된 checksum과 재계산한 checksum 비교

```
async verifyIntegrity(id: number): Promise<VerifyResult> {
  const { rows } = await this.db.query(
    'SELECT * FROM audit_log WHERE id = $1', [id],
  );
  if (rows.length === 0) throw new Error(`AuditLog not found: ${id}`);

  const row = rows[0!];
  const storedChecksum = row['checksum'] as string;
  const computedChecksum = this.generateChecksum(
    new Date(row['event_time'] as string),
    row['actor'] as string,
    row['action'] as string,
    row['resource_id'] as string,
    JSON.parse(row['after_state'] as string),
  );

  return { id, valid: storedChecksum === computedChecksum, storedChecksum, computedChecksum };
}
```

핵심 로직 — DB에서 읽은 값으로 checksum을 다시 계산해 저장된 값과 비교. 누군가 after_state나 actor를 수정했다면 재계산 결과가 달라져 **valid: false** 반환.

TypeScript → Java 호출 구조

FINALIZED 이후 Node.js가 HTTP로 Java internal-ledger에 두 가지를 위임한다

Node.js (TypeScript)

LedgerService.updateMintRequest()

→ status: FINALIZED

ICoreBankingAdapter (interface)

KyoboCoreBankingAdapter

InternalGatewayClient

POST

/users/{userId}
/nft-holdings

POST

/audit-log



HTTP
내부망

X-Internal
-Secret

Java (Spring Boot)

BlockchainGatewayController

InternalLedgerService

중복 방지 → save()

NftHoldingRepository

AuditLogService

REQUIRES_NEW TX

AuditLogRepository

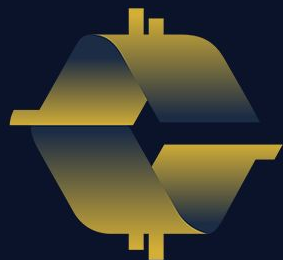
영구 원장 DB (개발: PostgreSQL / 운영: 교보 내부망 DB)

두 가지 위임: ① NFT 보유 이력 → `user_nft_holdings` INSERT | ② 감사 로그 → `audit_log` append-only INSERT



issuer-service

TxTransitionBridge · 발행 흐름 · WalletMappingService ·
IssuancePolicyService · IssuerService

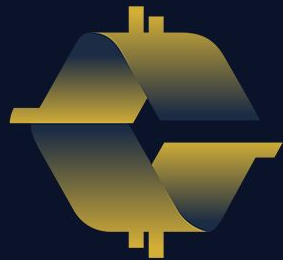


COINCRAFT
ACADEMY



issuer-service

TxTransitionBridge · 발행 흐름 · WalletMappingService ·
IssuancePolicyService · IssuerService



COINCRAFT
ACADEMY

두 개의 식별자 공간

두 세계 · 지갑 프로비저닝 정의

교보 내부 세계

```
userId = "K-20240001"  
// SSO 기반 고객 ID  
// 사람이 읽을 수 있음  
// 고객임을 식별 가능
```

교보생명 내부 인증 시스템이 부여하는 식별자. 블록체인에서는 아무 의미 없음.

블록체인 세계

```
walletAddr =  
"0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266"  
// ECDSA 공개키의 Keccak-256 해시  
// NFT는 이 주소로만 발행 가능
```

익명성. 주소만 보고는 누구인지 알 수 없음.

K-20240001

provision()

0xf39Fd6e5...

지갑 프로비저닝 = 두 식별자를 최초 1회 연결해서 DB에 저장하는 작업.
이후 NFT 발행 시 DB만 조회한다. VASP를 매번 호출하지 않는다.

user_wallet_mapping 테이블 설계 1/2

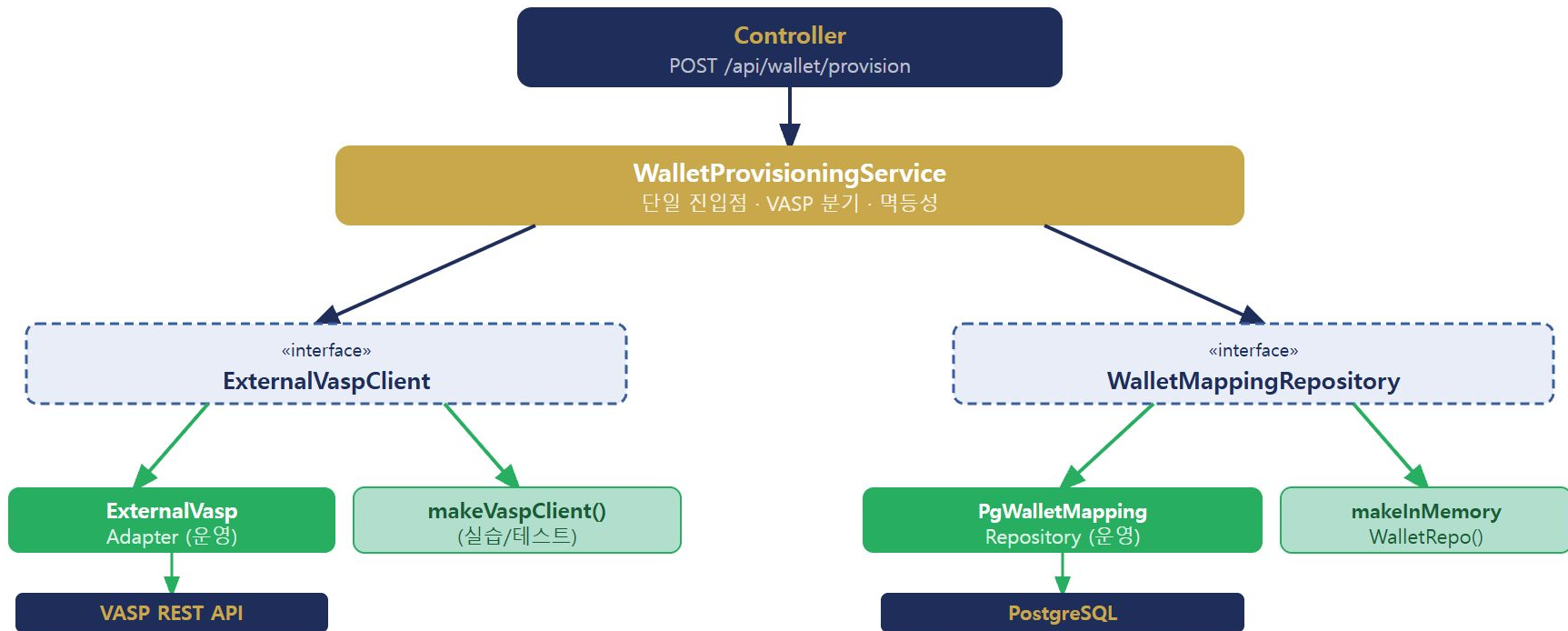
DB 스키마

```
CREATE TABLE user_wallet_mapping (  
  id          BIGSERIAL    PRIMARY KEY,  
  user_id     VARCHAR(64)  NOT NULL UNIQUE,  -- UNIQUE: userId당 1개 지갑만 허용 (Phase 1)  
  wallet_addr VARCHAR(42)  NOT NULL,         -- Ethereum 주소: 0x + 40자  
  vasp_type   VARCHAR(16)  NOT NULL,         -- 'EXTERNAL' | 'KYOBO'  
  verified    BOOLEAN      NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),  
  CONSTRAINT chk_vasp_type CHECK (vasp_type IN ('EXTERNAL', 'KYOBO'))  
);  
  
-- 지갑 주소로 역방향 조회 인덱스 (누가 이 지갑을 소유하는지)  
CREATE INDEX idx_wallet_mapping_addr  
ON user_wallet_mapping(wallet_addr);
```

UNIQUE(user_id) — DB 레벨에서 1:1 강제. 코드 레벨만으로는 부족하다 — 동시 요청 두 개가 동시에 통과할 수 있다 (TOCTOU).

전체 의존 관계 1/2

계층 구조 순서도



WalletMappingService 1/4

Repository 패턴 · interface WalletMappingRepository

서비스 코드에 SQL이 없다.
M4 AuditLogService 와 동일한 원칙.

```
interface WalletMappingRepository {  
  
  findByUserId(  
    userId: string  
  ): Promise<WalletMapping | null>;  
  
  save(  
    mapping: WalletMapping  
  ): Promise<void>;  
  
  findByWalletAddr(  
    walletAddr: string  
  ): Promise<WalletMapping | null>;  
}
```

WalletMapping 모델

```
interface WalletMapping {  
  userId: string;  
  walletAddr: string;  
  vaspType: VaspType;  
  verified: boolean;  
  createdAt: Date;  
}
```

3개 메서드의 역할

메서드	호출 시점
findByUserId	역등성 체크, 발행 전 주소 조회
save	프로비저닝 완료 후 저장
findByWalletAddr	역방향 조회 (감사, 분쟁)

WalletProvisioningService 2/4

provision() 내부 분기 — 생성자

```
export class WalletProvisioningService {  
  
  constructor(  
    private readonly vaspClient:  
      ExternalVaspClient,  
    private readonly walletMapping:  
      WalletMappingService,  
    private readonly auditLog?:  
      AuditLogAdapter,  
  ) {}  
  
  async provision(  
    userId: string,  
    vaspType: VaspType | string,  
  ): Promise<ProvisionResult> { ... }  
}
```

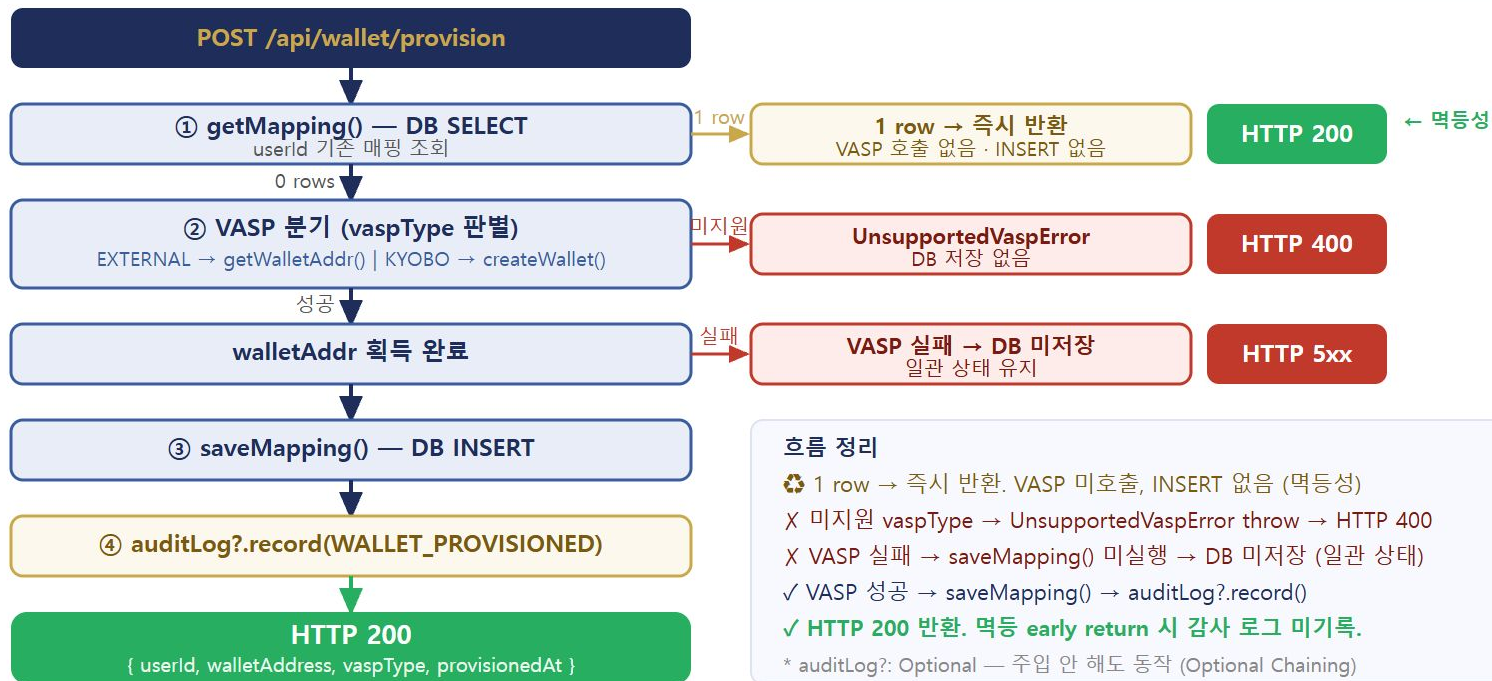
생성자 3개 파라미터

- 1 **vaspClient** (필수)
ExternalVaspClient 인터페이스 — VASP 호출
- 2 **walletMapping** (필수)
WalletMappingService — DB 저장·조회
- 3 **auditLog?** (선택적)
AuditLogAdapter — 감사 로그. 주입 안 해도 동작.

VaspType | string — string도 허용해서
미지원 타입 입력 시 UnsupportedVaspError를 throw할 수
있게 함.

provision() 완전한 흐름 정리

POST /api/wallet/provision { userId: 'K-20240001', vaspType: 'EXTERNAL' }



발행 전에 답해야 할 두 가지 질문

NFT 발행을 결정하는 두 개의 독립 관문

이벤트 · 사용자가 1만보를 걸었다 · 사용자가 건강검진을 받았다 · 사용자가 쿠폰을 사용했다

질문 1

이 이벤트에 대해 NFT를 발행할 수 있는가?

→ **IssuancePolicyService** 담당

- 정책 존재 여부
- 기간 유효성
- 수량 한도

질문 2

이 사용자가 조건을 충족하는가?

→ **EventConditionService** 담당

- 실제 이벤트 데이터 검증
- 사용자별 충족 여부

발행 정책 서비스 — DB 기반 관리 (2/4)

issuance_policy 테이블 설계

```
CREATE TABLE issuance_policy (  
  id          BIGSERIAL    PRIMARY KEY,  
  token_id    BIGINT       NOT NULL UNIQUE,  
  event_type  VARCHAR(64)  NOT NULL UNIQUE,  
  name        VARCHAR(128) NOT NULL,  
  max_per_user INT         NOT NULL DEFAULT 1,  
  total_supply INT,        -- NULL = 무제한  
  start_date  TIMESTAMPTZ  NOT NULL,  
  end_date    TIMESTAMPTZ  NOT NULL,  
  metadata_uri VARCHAR(512) NOT NULL,  
  is_active   BOOLEAN      NOT NULL DEFAULT true,  
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()  
);  
CREATE INDEX idx_policy_event_type ON issuance_policy (event_type);
```

UNIQUE(event_type) — 이벤트 타입당 정책은 하나. token_id도 UNIQUE — 정책마다 고유 NFT 타입.

발행 정책 서비스 — DB 기반 관리 (4/4)

PgIssuancePolicyRepository — 조회 쿼리

findByEventType

```
async findByEventType(
  eventType: string,
): Promise<IssuancePolicy | null> {
  const res = await this.pool.query(
    `SELECT id, token_id, event_type, name,
      max_per_user, total_supply,
      start_date, end_date,
      metadata_uri, is_active
    FROM issuance_policy
    WHERE event_type = $1`,
    [eventType],
  );
  if (!res.rowCount) return null;
  return this._toModel(res.rows[0]);
}
```

is_active + 기간 검증 (서비스 레이어)

```
async getPolicy(
  eventType: string,
): Promise<IssuancePolicy> {
  const p = await this.repo
    .findByEventType(eventType);

  if (!p || !p.isActive)
    throw new NoPolicyError(eventType);

  const now = new Date();
  if (now < p.startDate || now > p.endDate)
    throw new NoPolicyError(eventType);

  return p;
}
```

조건 판단 — Strategy 패턴

정책이 "무엇을" 결정한다면, 조건은 "할 수 있는가"를 결정

정책 (IssuancePolicyService)

- 이 이벤트의 tokenId는?
- 기간이 유효한가?
- maxPerUser는?

조건 (EventConditionChecker)

- 이 사용자가 실제로 1만보를 걸었는가?
- 검진을 받았는가?
- 쿠폰이 유효한가?

오케스트레이터 (IssuerService)

정책 통과? + 조건 통과?
→ 발행

Strategy 패턴 핵심 아이디어

각 이벤트의 조건 판단 로직을 별도 클래스(Checker)로 캡슐화한다. EventConditionService는 Checker 목록을 받아 eventType에 맞는 것을 찾아 위임한다.

새 이벤트 추가 = 새 Checker 클래스 작성 + 등록. 기존 코드 수정 없음 (OCP)

세 상태 공간 — TX 8개 · MintRequest 7개 · IssuanceRequest 4개

TxTransitionBridge가 세 공간을 직렬 연결한다 — 각 상태 공간은 독립적으로 보호됨

TxStatus (8개) — tx_mint_requests

REQUESTED — 발행 요청 수신, TX 미생성

SUBMITTED — TX 브로드캐스트 완료

PENDING — mempool 체류 (타임아웃 감시)

MINED — 블록 포함 확인

CONFIRMED — N블록 확정

FINALIZED — 내부 원장 정산 완료

FAILED — revert / 영구 실패

REORGED — 체인 롤백 감지

체인 외부 상태 — VASP 콜백·블록 풀링 트리거

MintStatus (7개) — mint_requests

REQUESTED

SUBMITTED

MINED

CONFIRMED

FINALIZED

FAILED

REORGED

내부 원장 — LedgerService 관리

IssuanceStatus (4개) — issuance_requests

REQUESTED — 요청 접수

SUBMITTED — VASP 요청 완료

CONFIRMED — 체인 확정

FAILED — 최종 실패

사용자 가시성 — Kyobo App이 읽는 최종 상태

TxTransitionBridge: TxStatus 전이 이벤트 수신 → MintStatus 갱신(LedgerService) → IssuanceStatus 갱신(IssuanceRepo) 직렬 연쇄. **PENDING·MINED·REORGED**는 TxStatus/MintStatus에만 존재 — IssuanceRequest는 사용자 관점 4단계(REQUESTED·SUBMITTED·CONFIRMED·FAILED)만 노출.

발행 흐름 Phase 1 — IssuerService 동기 구간

issueActivityNFT() 반환 전까지 순서 보장 — 세 테이블에 REQUESTED-SUBMITTED 기록

#	트리거 (메서드)	테이블	상태 변화
①	issuanceRepo.create()	issuance_requests	REQUESTED 생성
②	ledgerService.createMintRequest()	mint_requests	REQUESTED 생성
③	txStateMachine.submitMintRequest() → repo.save()	tx_mint_requests	REQUESTED 생성
④	vasp.submitMint() 성공 → _transition('SUBMITTED') txHash 최초 획득	tx_mint_requests	REQUESTED → SUBMITTED
⑤	issuanceRepo.updateStatus('SUBMITTED') ← IssuerService 직접 호출	issuance_requests	REQUESTED → SUBMITTED
⑥	ledgerService.updateMintRequest({ status:'SUBMITTED', txHash }) ← 직접 호출	mint_requests	REQUESTED → SUBMITTED
⑦	[선택적 — TX mempool 체류 시] pollStaleRequests() / handleTimeout() — STALE_MINUTES 초과 감지. TX가 결국 채굴되면 Phase 2A ⑦로 진입	tx_mint_requests	SUBMITTED → PENDING Bridge 스킵 — mint/issuance 불변

⑤⑥은 TxTransitionBridge가 아닌 IssuerService가 직접 쓴다 — SUBMITTED 전이 이벤트 시점에 txHash가 아직 mint_requests에 없어 Bridge가 findByTxHash()로 찾지 못하기 때문.
여기서 issueActivityNFT() 반환.

발행 흐름 Phase 2A — VASP 콜백 경로 (비동기)

NFT_ISSUED Webhook → Redis Stream → NFTIssuedProcessor → TxTransitionBridge 연쇄

#	트리거 (메서드)	테이블	상태 변화
↪	[타임아웃 우회 — ⑦ 이전] TX mempool 채류 중 pollStaleRequests() / handleTimeout() 감지 → TX가 결국 채굴되면 NFT_ISSUED 발행 후 ⑦로 진입	tx_mint_requests	SUBMITTED → PENDING Bridge 스킵 — mint/issuance 불변
⑦	TX 채굴 → VASPServer NFT_ISSUED 발행 → webhook → WebhookServer._handleRequest()	—	수신·즉시 202
⑧	WebhookPublishHandler → RedisStreamPublisher.publish()	Redis Stream	XADD
⑨	ConsumerGroupWorker → NFTIssuedProcessor.process() → updateMintRequestConfirmed() → txStateMachine.handleMined()	tx_mint_requests	SUBMITTED or PENDING → MINED
⑩	handleMined._transition → TxTransitionBridge → ledgerService.updateMintRequest('MINED')	mint_requests	SUBMITTED → MINED
⑪	txStateMachine.handleConfirmed()	tx_mint_requests	MINED → CONFIRMED
⑫	handleConfirmed._transition → TxTransitionBridge → ledgerService.updateMintRequest('CONFIRMED')	mint_requests	MINED → CONFIRMED
⑬	TxTransitionBridge → issuanceRepo.updateStatus('CONFIRMED')	issuance_requests	SUBMITTED → CONFIRMED ✓

tx_mint_requests가 항상 먼저 — TxTransitionBridge가 비동기로 mint_requests → issuance_requests 순으로 연쇄. 찰나 순간 세 테이블 상태가 다를 수 있어 InvalidStateTransitionError catch로 막등 처리.

발행 흐름 Phase 2B — ChainEventListener 폴백 (no-emit)

VASP 콜백 미도착 시 블록 직접 폴링 → IssuanceConfirmHandler → TxTransitionBridge 연쇄

#	트리거 (메서드)	테이블	상태 변화
⑦'	ChainEventListener 블록 폴링 → EVMAdapter.queryEvents('Issued') 감지	—	이벤트 발견
⑧'	IssuanceConfirmHandler.handle(event) → txRepo.findByTxHash() → txStateMachine.handleMined()	tx_mint_requests	SUBMITTED or PENDING → MINED
⑨'	handleMined._transition → TxTransitionBridge → ledgerService.updateMintRequest('MINED')	mint_requests	SUBMITTED → MINED
⑩'	txStateMachine.handleConfirmed()	tx_mint_requests	MINED → CONFIRMED
⑪'	handleConfirmed._transition → TxTransitionBridge → ledgerService.updateMintRequest('CONFIRMED')	mint_requests	MINED → CONFIRMED
⑫'	TxTransitionBridge → issuanceRepo.updateStatus('CONFIRMED')	issuance_requests	SUBMITTED → CONFIRMED ✓

2A vs 2B 차이: 2A는 Webhook→Redis→Consumer 경로. 2B는 블록 직접 폴링. ⑧'~⑫' 구조는 2A의 ⑨~⑬과 동일 — IssuanceConfirmHandler가 handleMined()+handleConfirmed()를 직접 호출해 같은 TxTransitionBridge 연쇄로 수렴.

TxTransitionBridge — EventEmitter 연결 구조

TxStateMachineService._transition() → emit('transition') → Bridge._handle() → 두 테이블 cascade



TX_TO_MINT 매핑 (Bridge 내부 상수)

TxStatus (emit)	mint_requests	issuance_requests
SUBMITTED	→ SUBMITTED	— (IssuerService 직접)
PENDING	매핑 없음 — skip	불변
MINED	→ MINED	불변
CONFIRMED	→ CONFIRMED	→ CONFIRMED ✓
FAILED	→ FAILED	→ FAILED
FINALIZED	→ FINALIZED	불변
REORGED	→ REORGED	불변

두 진입점 → 동일 cascade

Phase 2A — NFT_ISSUED Webhook → Redis → NFTIssuedProcessor
→ handleMined()+handleConfirmed() → _transition() → Bridge

Phase 2B — 블록 직접 폴링 → IssuanceConfirmHandler
→ handleMined()+handleConfirmed() → _transition() → Bridge

진입점이 달라도 _transition() → emit으로 수렴 — cascade 로직 중복 없음

SUBMITTED 예외 — submitMintRequest() 반환 전 txHash가 아직 mint_requests에 없음 → Bridge findByTxHash() 미탐 → IssuerService가 ⑤⑥ 직접 씬

역동 처리 — MINED-CONFIRMED 이벤트 concurrent 발화 시 InvalidStateTransitionError catch → 선행 처리된 상태 그대로 진행

제어권 이전 경계 — submitMintRequest() 반환 시점 : 그 전(Phase 1)은 비즈니스 로직(IssuerService)이 세 테이블을 직접 주도. 그 후(Phase 2)는 TxStateMachineService가 온체인 이벤트를 받아 _transition() → emit → Bridge cascade로 자동 전이.

Strategy 패턴 — 공통 인터페이스 (1/5)

EventConditionChecker · EvaluationContext — 모든 Checker가 구현하는 계약

EvaluationContext

```
interface EvaluationContext {  
  userId:    string;  
  eventType: string;  
  payload:   unknown;  
}
```

EventConditionChecker

```
interface EventConditionChecker {  
  /** 이 Checker가 eventType을 처리할 수 있는가 */  
  supports(eventType: string): boolean;  
  /** 조건 충족 여부 반환 */  
  check(ctx: EvaluationContext): Promise<boolean>;  
}
```

supports()가 라우팅을 결정한다. EventConditionService는 supports()=true인 Checker에게 check()를 위임한다.

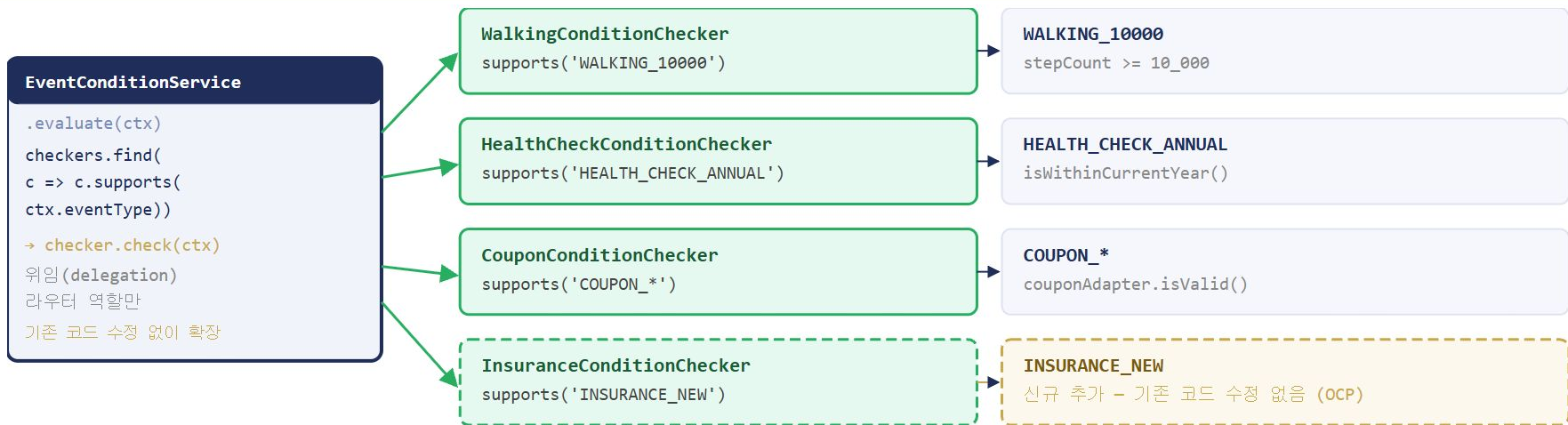
두 메서드의 역할 분리

메서드	역할
supports()	내가 이 이벤트를 처리할 수 있는가? (라우팅)
check()	실제 조건 평가 (비즈니스 로직)

payload: unknown — 각 Checker가 자신이 아는 타입으로 캐스팅. 외부로 타입 의존성 노출 없음.

적용 후 — Strategy 패턴

각 Checker가 독립적으로 존재. EventConditionService는 라우터만



OCP 달성

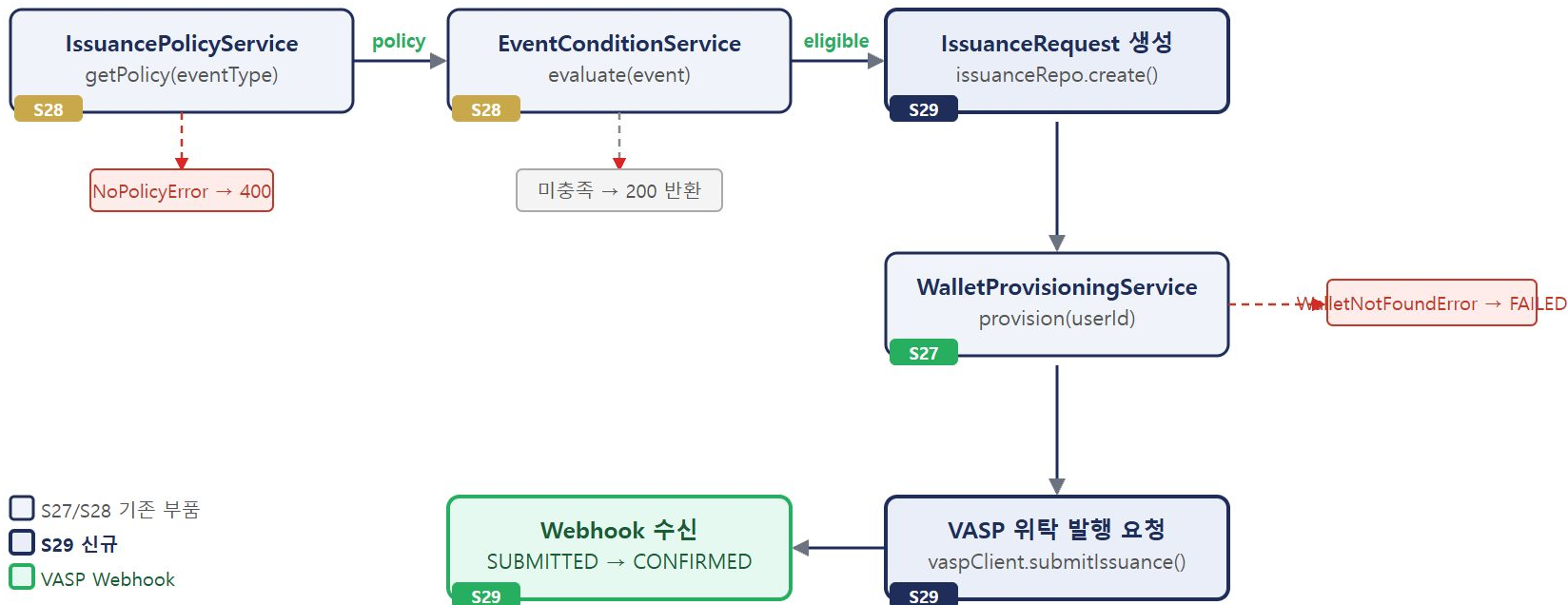
새 이벤트: 파일 1개 추가 + DI 등록 1줄. 기존 코드 수정 없음.

테스트 격리

각 Checker를 독립 단위 테스트. Mock 없이도 순수 로직 검증 가능.

S27~S28에서 만든 부품이 여기서 연결된다

IssuerService — 오케스트레이터. 직접 DB를 건드리지 않는다.



IssuanceRequest — 발행 요청 상태 엔티티

NFT 발행은 VASP에 위탁하는 순간 비동기가 된다 — 상태 추적이 필요하다

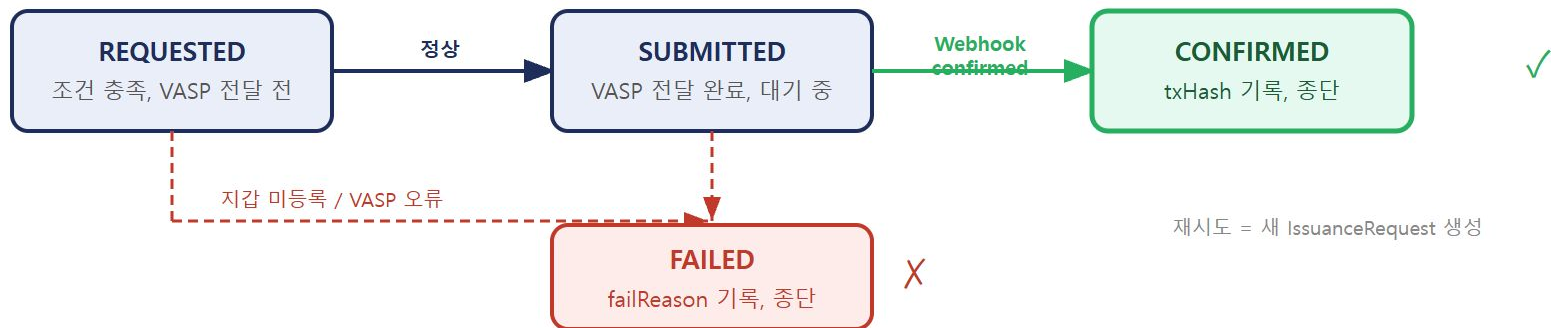
```
export type IssuanceStatus =  
  'REQUESTED' // 조건 통과, VASP 전달 전  
  | 'SUBMITTED' // VASP 전달 완료, 대기  
  | 'CONFIRMED' // Webhook 수신, 종단  
  | 'FAILED'; // 어느 단계든 실패, 종단
```

```
export interface IssuanceRequest {  
  id: string; // UUID  
  userId: string;  
  eventType: string; // 'WALK_GOAL_MET'  
  tokenId: bigint; // 생성 시 즉시 확정  
  amount: number;  
  walletAddr: string | null; // 지갑 조회 전 null  
  status: IssuanceStatus;  
  txHash: string | null; // CONFIRMED 후 기록  
  failReason: string | null;  
  createdAt: Date;  
  updatedAt: Date;  
}
```

mint_requests(M4)와 다른 점 · tokenId가 생성 시 즉시 확정 (M4는 CONFIRMED 이후) · walletAddr-amount 포함 · blockNumber-retryCount 없음

IssuanceRequest — 상태 흐름

4개 상태 — VASP Webhook 기준으로 전이한다



상태	의미	누가 전이	DB 변화
REQUESTED	조건 충족, VASP 전달 전	IssuerService.issueActivityNFT()	INSERT, walletAddr 채워짐
SUBMITTED	VASP 전달 완료, 대기	IssuerService.issueActivityNFT()	status 갱신
CONFIRMED	온체인 TX 확정 — 종단	IssuanceConfirmHandler	txHash 채워짐
FAILED	파이프라인 임의 단계 실패, 종단	IssuerService / IssuanceConfirmHandler	failReason 채워짐

IssuanceRequest 리포지토리 인터페이스

IssuerService는 이 인터페이스에만 의존한다 — 운영: Postgres / 테스트: InMemory

```
export interface IssuanceRequestRepository {  
  // 신규 발행 요청 생성  
  create(  
    req: Omit<IssuanceRequest, 'id' | 'createdAt' | 'updatedAt'>  
  ): Promise<IssuanceRequest>;  
  
  // ID로 단건 조회  
  findById(id: string): Promise<IssuanceRequest | null>;  
  
  // 상태 전이 + 추가 필드 업데이트  
  updateStatus(  
    id: string,  
    status: IssuanceStatus,  
    extra?: {  
      txHash?: string;  
      walletAddr?: string;  
      failReason?: string;  
    },  
  ): Promise<void>;  
}
```

인터페이스만 정의하면 된다

운영에서는 `PgIssuanceRequestRepository`

테스트에서는 `makeInMemoryIssuanceRepo()`

updateStatus 불변 조건

CONFIRMED 또는 FAILED 상태에서 전이 시도 → 예러

상태 전이 불가: CONFIRMED → REQUESTED

setWalletAddr 별도 메서드 고려

지갑 주소 업데이트는 상태 전이가 아닌 데이터 보정.

`updateStatus(id, 'REQUESTED', { walletAddr })` 는

동일 상태 전이 → 가드 로직과 충돌 가능

IssuerService — 의존 관계

의존성 목록이 곧 서비스의 책임 목록이다

```
export class IssuerService {  
  constructor(private readonly deps: {  
    policyService: IssuancePolicyService;  
    conditionService: EventConditionService;  
    walletProvisioning: WalletProvisioningService;  
    issuanceRepo: IssuanceRequestRepository;  
    vaspClient: IVaspIssuanceClient;  
  }) {}  
}
```

의존성	역할	위치
policyService	tokenId-amount 조회	S28
conditionService	조건 충족 여부 판단	S28
walletProvisioning	사용자 지갑 주소 조회	S27
issuanceRepo	발행 요청 상태 기록	S29
vaspClient	VASP에 발행 위탁	S29

IssuerService는 직접 DB를 건드리지 않는다

모든 의존성이 인터페이스다. EVM → XRPL로 체인이 바뀌어도 IssuerService 코드는 변경 없다.

IVaspIssuanceClient 인터페이스

VASP 위탁 발행 — 비동기 응답 모델

```
export interface IVaspIssuanceClient {  
  submitIssuance(params: {  
    requestId: string;  
    walletAddr: string;  
    tokenId: bigint;  
    amount: number;  
  }): Promise<{  
    submitted: boolean;  
    vaspRefId: string;  
  }>;  
}
```

비동기 응답 모델

`submitIssuance()` 는 "요청을 접수했다"는 응답만 반환.
체인 확정 결과는 VASP → Webhook으로 별도 수신.

VASP 위탁 흐름

- ① submitIssuance() 호출 → VASP 접수
- ② IssuanceStatus: REQUESTED → SUBMITTED
- ③ VASP 내부에서 TX 서명·브로드캐스트
- ④ 체인 확정 후 Webhook 수신
- ⑤ IssuanceStatus: SUBMITTED → CONFIRMED

인터페이스에만 의존하는 이유

교보생명 VASP(월렛원)가 바뀌어도 `IVaspIssuanceClient` 구현체만 교체하면 된
다. IssuerService 코드는 변경 없음.

issueActivityNFT() — 6단계 파이프라인 (1 / 4)

함수 선언부 · ① 발행 정책 조회

```
async issueActivityNFT(event: {
  userId:    string;
  eventType: string;
  data:      Record<string, unknown>;
}): Promise<{ requestId: string; eligible: boolean }> {
  const { userId, eventType, data } = event;

  // ① 발행 정책 조회
  const policy = await this.deps.policyService.getPolicy(eventType);
  // NoPolicyError 발생 시 상위로 전파 → 400
  // policy: { tokenId, amount, validFrom, validTo }
```

getPolicy()가 실패하면 · `NoPolicyError` throw → `issueActivityNFT()` 밖으로 전파 → 컨트롤러에서 400 응답 · `issuance_request` 레코드 생성 안 됨

`policyService.getPolicy()`는 DB를 읽는다 (`issuance_policy` 테이블). `IssuerService`는 직접 DB 접근하지 않고 `policyService`를 통해 간접 접근.

issueActivityNFT() — 6단계 파이프라인 (2 / 4)

② 조건 판단 · ③ 발행 요청 생성 (REQUESTED)

```
// ② 조건 판단
const condition = await this.deps.conditionService.evaluate({ userId, eventType, data });
if (!condition.eligible) {
  return { requestId: '', eligible: false };
  // 조건 미충족 → 발행 요청 자체를 생성하지 않는다
}

// ③ 발행 요청 생성 (REQUESTED)
const req = await this.deps.issuanceRepo.create({
  userId, eventType,
  tokenId: policy.tokenId, // 정책에서 가져온 값 - 즉시 확정
  amount: policy.amount,
  walletAddr: null, // 지갑 조회 전 null
  status: 'REQUESTED',
  txHash: null,
  failReason: null,
});
```

② eligible: false이면 ③ 실행 안 함

조건 미충족은 에러가 아니라 정상 흐름. 레코드가 없어야 맞다. 재처리 대상이 아님을 명확히.

③ tokenId가 생성 시 확정되는 이유

정책 조회 결과가 이미 있다. MintStatus(mint_requests)와 달리 CONFIRMED까지 기다릴 필요 없음.

issueActivityNFT() — 6단계 파이프라인 (3 / 4)

④ 지갑 주소 조회

```
// ④ 지갑 주소 조회
let walletAddr: string;
try {
  const result = await this.deps.walletProvisioning.provision(userId, 'EXTERNAL');
  walletAddr = result.walletAddress;
  // walletAddr 보정 - 상태 전이 없음, 데이터 업데이트만
  await this.deps.issuanceRepo.setWalletAddr(req.id, walletAddr);
} catch (err) {
  await this.deps.issuanceRepo.updateStatus(req.id, 'FAILED', {
    failReason: (err as Error).message,
  });
  throw err;
  // WalletNotFoundError → 상위(Worker)가 DLQ로 이동
}
```

setWalletAddr() 별도 메서드

지갑 주소 업데이트는 상태 전이가 아닌 데이터 보정. `updateStatus(id, 'REQUESTED', { walletAddr })` 는 동일 상태 전이로 가드에서 에러 발생 가능.

지갑 조회 실패 → FAILED + throw

`issuance_request`에 FAILED를 기록하고 에러를 다시 던진다. 어느 단계에서 실패했는지 DB에서 즉시 파악 가능.

issueActivityNFT() — 6단계 파이프라인 (4 / 4)

⑤ VASP 위탁 발행 요청 · ⑥ SUBMITTED 상태로 반환

```
// ⑤ VASP 위탁 발행 요청
try {
  await this.deps.vaspClient.submitIssuance({
    requestId: req.id,
    walletAddr,
    tokenId:    policy.tokenId,
    amount:    policy.amount,
  });
  await this.deps.issuanceRepo.updateStatus(req.id, 'SUBMITTED');
} catch (err) {
  await this.deps.issuanceRepo.updateStatus(req.id, 'FAILED', {
    failReason: (err as Error).message,
  });
  throw err;
}

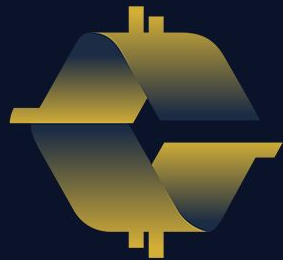
// ⑥ SUBMITTED 상태로 반환 - 체인 확인은 Webhook에서 처리
return { requestId: req.id, eligible: true };
}
```

issueActivityNFT()은 **CONFIRMED**를 직접 설정하지 않는다 · VASP에 요청을 보내고 SUBMITTED를 기록한 뒤 즉시 반환. 체인 확인은 나중에 Webhook 핸들러가 처리.

VI

Phase 1 완성 검증

두 발행 경로 · ChainEventListener 풀백 · 통합 테스트 10개 시나리오



COINCRAFT
ACADEMY

두 발행 경로 — 정상 경로 vs 폴백 경로

Phase 1은 두 경로를 동시에 가동한다 — VASP 장애·no-emit에서도 CONFIRMED 보장

① VASP 콜백 경로 (정상)

```
TX 브로드캐스트 (ExternalVASPAdapter)
↓
VASP 서버 - Issued 이벤트 감지
↓
NFT_ISSUED Webhook → POST :19877
↓
WebhookPublishHandler → Redis XADD
↓
NFTIssuedProcessor.process()
↓
LedgerService.updateMintRequest(CONFIRMED)
↓ TxTransitionBridge
issuance_requests → CONFIRMED ✓
```

정상 발행 시 이 경로가 먼저 도착.

VASP가 체인 이벤트를 인지하고 콜백을 보내는 구조.

② ChainEventListener 폴백 경로

```
TX 브로드캐스트 (ExternalVASPAdapter)
↓
Issued 이벤트 미발행(no-emit)
또는 VASP 콜백 미도착
↓
ChainEventListener - 블록 폴링
  EVMAAdapter.queryEvents('Issued')
↓
IssuanceConfirmHandler.handle(event)
↓
TxStateMachineService
  handleMined() → MINED
  handleConfirmed() → CONFIRMED
↓ TxTransitionBridge
issuance_requests → CONFIRMED ✓
```

VASP 콜백 없을 때 복구 경로.

블록을 직접 폴링해 Issued 이벤트를 찾아 처리.

ChainEventListener + IssuanceConfirmHandler 연결 구조

온체인 이벤트 → TxStateMachineService 경유 → 세 Status 동시 동기화

ChainEventListener 기동 (index.ts)

```
const eventListener = new ChainEventListener(
  chainAdapter,
  [issuanceConfirmHandler, // Issued → CONFIRMED 전이
    new ProcessedEventHandler(...)], // 먹등성 기록
  [{ addr: NFT_CONTRACT_ADDR,
    eventNames: ['Issued'] }],
  { getLastProcessedBlock,
    setLastProcessedBlock }, // 재시작 내성
);
```

IssuanceConfirmHandler.handle()

```
async handle(event: ChainEvent): Promise<void> {
  const txReq = await this.txRepo.findByTxHash(event.txHash);
  if (!txReq) return; // 관련 없는 TX 스킵

  // 먹등 가드 - 이미 처리된 TX 재도착 시 무시
  if (txReq.status === 'CONFIRMED' ||
    txReq.status === 'FINALIZED') return;

  if (txReq.status === 'SUBMITTED' ||
    txReq.status === 'PENDING')
    await this.txStateMachine.handleMined(txReq.id, event.blockNumber);
```

상태 전이 연쇄

```
ChainEventListener
  → Issued 이벤트 감지 (txHash + blockNumber)
  ↓
IssuanceConfirmHandler.handle()
  → txRepo.findByTxHash(txHash)
  ↓
TxStateMachineService.handleMined()
  → tx_mint_requests: SUBMITTED → MINED
  → TxTransitionBridge 'transition' 이벤트
    → mint_requests: SUBMITTED → MINED
  ↓
TxStateMachineService.handleConfirmed()
  → tx_mint_requests: MINED → CONFIRMED
  → TxTransitionBridge 'transition' 이벤트
    → mint_requests: MINED → CONFIRMED
    → issuance_requests → CONFIRMED ✓
```

단일 이벤트 흐름으로 세 Status 동기화

TxStatus → MintStatus → IssuanceStatus 순서로 자동 연쇄.
IssuanceConfirmHandler는 issuance_requests를 직접 건드리지 않는다.

no-emit 시나리오 — 폴백 경로 타임라인

mint() TX는 성공했지만 Issued 이벤트를 emit하지 않는 경우 — ChainEventListener가 복구

- 1 **MockVASP.setMode(NO_EMIT)** — mint() TX를 브로드캐스트하지만 emit Issued()를 실행하지 않는다
- 2 **VASP 콜백 없음** — Issued 이벤트 미발행 → VASPServer가 NFT_ISSUED webhook을 보내지 않음 → NFTIssuedProcessor 미호출
- 3 **tx_mint_requests: SUBMITTED 유지** — TX 채굴이 완료됐지만 상태 전이 없음. issuance_requests도 SUBMITTED에 멈춤.
- 4 **ChainEventListener 블록 폴링** — EVMAAdapter.queryEvents('Issued')로 블록을 직접 조회. mint() TX가 포함된 블록에서 Issued 이벤트를 찾음.
- 5 **IssuanceConfirmHandler → 상태 전이 연쇄** — txHash로 tx_mint_requests 조회 → handleMined() → handleConfirmed() → TxTransitionBridge → **CONFIRMED** ✓

no-emit이 테스트 시나리오인 이유

실제 운영에서 VASP 서버 버그 또는 네트워크 단절로 콜백이 누락될 수 있다.

ChainEventListener 폴백이 없으면 issuance_requests가 SUBMITTED에 영원히 멈춘다.

Hardhat no-emit 재현 방법

MockVASP.setMode(NO_EMIT) → TX 채굴은 되지만 이벤트 없음

DB polling으로 tx_hash 확인 후 setMode(NORMAL) 복원 (race condition 방지)

VASPServer — 통합 테스트 인프라

실제 외부 VASP 없이 전체 파이프라인을 테스트할 수 있게 해주는 핵심 헬퍼

역할

① HTTP 서버 (:19876)

ExternalVASPAdapter가 POST /transactions를 보내면 실제로 수신한다. 요청을 받으면 MockVASP 컨트랙트에 온체인 TX를 브로드캐스트한다.

② 콜백 발신

TX 확정 후 issuer-service WebhookServer(:19877)에 NFT_ISSUED 콜백을 보낸다. 실제 VASP의 webhook 발신 동작을 재현.

③ NonceManager + resetNonce()

ethers NonceManager로 nonce를 로컬 추적. Sepolia 테스트 beforeEach에서 resetNonce()를 호출해 이전 테스트 PEL 재시도로 인한 nonce 불일치를 방지.

MockVASP 모드

모드	동작	테스트 대상
NORMAL	mint() 성공 + Issued emit	[1] 정상 발행
REVERT	estimateGas 단계에서 revert	[2] TX revert
NO_EMIT	mint() 성공 · Issued 미발행	[3] no-emit 폴백

```
// 테스트 기동
vaspServer = new VASPServer({
  port: 19876,
  rpcUrl: hardhatRpc,
  operatorKey: OPERATOR_KEY,
  contractAddr: MOCK_VASP_ADDR,
  callbackUrl: 'http://localhost:19877',
});
await vaspServer.start();

// beforeEach: DB 초기화 + nonce 재동기화
vaspServer?.resetNonce();
```


통합 테스트 — 10개 시나리오 (mock-vasp)

Phase 1 완성 기준 — 10개 전체 통과 시 파이프라인 검증 완료

케이스	시나리오	검증 대상	최종 상태
[1] NORMAL	정상 발행 → VASPServer NFT_ISSUED 콜백 → Redis Stream → ledger	정상 E2E 전체 경로	CONFIRMED
[2] REVERT	MockVASP REVERT 모드 → estimateGas 단계 revert → VASPServer 500	TX revert 처리	FAILED
[3] NO_EMIT	mint() 성공 · Issued 이벤트 미발행 → ChainEventListener 폴백	폴백 경로	SUBMITTED 유지
[4] PENDING	블록 채굴 중단 → TX mempool 체류 → mineBlock → CONFIRMED	mempool 대기 처리	CONFIRMED
[5] REORG	snapshot → 발행 → revertToSnapshot → 온체인 TX 소실	체인 재편성 복구	REORGED
[stream-2]	동일 requestId 중복 주입 → 멍등성 보장 (한 번만 처리)	Redis Stream 멍등성	중복 무시
[stream-3]	retryCount ≥ 3 → DLQ 이동 + 원본 Stream XACK	DLQ 이동 조건	DLQ 이동
[stream-4]	Consumer crash → PEL 잔류 → minIdleMs 초과 → XAUTOCLAIM	PEL 자동 복구	재처리 성공
[poll-1]	SUBMITTED → PENDING 직접 주입 → pollStaleRequests() 실행 → CONFIRMED	stale TX 강제 복구	CONFIRMED
[burst]	동시 5명 웹훅 전송 → 병렬 파이프라인 → 전체 CONFIRMED	동시성 처리	전체 CONFIRMED

npm run test:mock-vasp — Tests: 10 passed, 10 total · Sepolia: 9 passed (burst 제외 — 실제 ETH 소모)

Phase 1 — 전체 구성 요약

구현 완료 컴포넌트 · 검증 방법 · Phase 2/3 전환 포인트

이벤트 수신 레이어

- WebhookServer (HMAC 검증)
- WebhookPublishHandler
- RedisStreamPublisher (XADD)
- IdempotencyGuard

처리 레이어

- ConsumerGroupPool
- NFTIssuedProcessor
- ActivityProcessor
- DLQHandler

발행 레이어

- IssuerService
- TxStateMachineService
- ExternalVASPAdapter
- VaspTxClientAdapter

폴백 / 복구 레이어

- ChainEventListener (블록 폴링)
- IssuanceConfirmHandler
- pollStaleRequests (Scheduler)
- TxTransitionBridge

원장 레이어

- LedgerService (mint_requests)
- AuditLogService (SHA-256)
- ReconcileService
- Java Gateway (영구 원장)

Phase 2/3 전환 포인트

- ExternalVASP → KyoboVASP
- ChainEventListener 상태 스토어
- Circle Arc USDC/KRW1 레이어
- HSM/MPC 직접 서명

Phase 1 완성 기준: mock-vasp 통합 테스트 10/10 통과 · Sepolia 통합 테스트 9/9 통과 · 전 시나리오 데모 실행 가능